



VELS





INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR, ADYAR - CHENNAI

Cognitive Evolution Engine (CEE): A Middleware Framework for Self-Evolving Physical AI Robots Using Reflective Knowledge Intelligence

FINAL YEAR

PROJECT PHASE-I REPORT

Submitted by

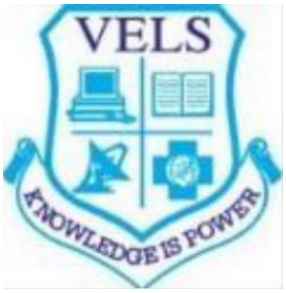
Swarna M (23619139)

Vishal S S M (23619143)

Aanandhasubash S (23619147)

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY CSE -
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
SCHOOL OF ENGINEERING
VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project “Cognitive Evolution Engine (CEE): A Middleware Framework for Self-Evolving Physical AI Robots Using Reflective Knowledge Intelligence” is the bonafide work of **Swarna M (23619139), Vishal S S M (23619143), Aanandhasubash S (23619147)** who carried out the project work under my supervision. Certified further that, to the best of my knowledge, the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD
Dr. K. Ulaga Priya, M.E., Ph.D.
Associate Professor & Head
Department of AI & ML
School of Engineering
VISTAS
Chennai - 600 117

SIGNATURE OF THE SUPERVISORS
Ms. Usha B M.Tech (Ph.D), Ms. Rekhadevi B M.Tech.
Assistant Professors
Department of AI & ML
School of Engineering
VISTAS
Chennai - 600 117

Submitted for the Project Work and Viva-Voce examination held on …… at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support I received from the positive minds around me in making this venture a successful one. The infrastructural support with all kinds of lab facilities have been a motivating factor in this completion of project work, all because of FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH., with great pleasure that I take this opportunity to thank him. I am grateful to our HEAD OF THE DEPARTMENT, Dr. K. Ulagapriya, for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day. I extend our warmest thanks to my INTERNAL GUIDE, Ms. Usha B, and Ms. Rekhadevi B who has patiently guided and provided us with invaluable advice and help. I am thankful to my Project Coordinator Ms. S. J Kavitha. I would like to express my greatest gratitude to our Staff members, Librarian, and Non-teaching staff members of Computer Centre, who have helped and supported me throughout. Last but not the Least, I wish to thank my parents and my friends for their individual support and interest who has inspired me and encouraged me to go in my own way, without them I would be unable to complete this project.

SWARNA M **23619139**

VISHAL S S M **23619143**

AANANDHASUBASH **23619147**

TABLE OF CONTENTS

ABSTRACT **7**
LIST OF FIGURES **8**
LIST OF TABLES **9**
LIST OF ABBREVIATIONS **10**
INTRODUCTION **11**
1.1 General 11
1.2 Problem Description 11
1.3 Existing System 12
1.4 Proposed System 12
1.5 Objectives 13
1.6 Scope of the Project 14
1.7 Contributions 14
1.8 Modules 15
1.9 Summary 15
LITERATURE REVIEW **16**
2.1 Continual and Lifelong Learning in Robotics 16
2.2 Persistent Memory and Cognitive Architectures 16
2.3 Knowledge Representation and Retrieval 17
2.4 Reflection and Closed-Loop Reasoning 17
2.5 Autonomous Exploration and Knowledge Transfer 17
2.6 Existing Systems and Limitations 18
2.7 Research Gap 18
2.8 Summary 19
SYSTEM DESIGN **20**
3.1 Existing System 20
3.2 Proposed System 20
3.3 System Architecture 21
3.4 Overall Workflow 23
3.5 System Modules 24
3.6 Data Flow 25
3.7 Hardware Requirements 26
3.8 Software Requirements 27
3.9 Summary 28
IMPLEMENTATION AND METHODOLOGY **29**
4.1 Mobile Robot Simulation 29
4.2 Telemetry Data Collection 29

4.3 MQTT-Based Communication....31
4.4 Telemetry Variable Processing....31
4.5 Condition-Based Reasoning....32
4.6 Operational Rule Mapping....33
4.7 Event Latching and Duplicate Suppression....34
4.8 Knowledge Repository and History....35
4.9 Dashboard Implementation....37
4.10 Telemetry Visualization....38
4.11 Testing Methodology....39
4.12 Summary....41
RESULTS AND DISCUSSION....43
5.1 Experimental Setup....43
5.2 Telemetry Monitoring Results....43
5.3 Condition-Based Event Detection....44
5.4 Operational Rule Generation....45
5.5 Event Latching Results....46
5.6 Knowledge History and Retention....47
5.7 Dashboard Visualization....48
5.8 Functional Testing....49
5.9 Observed Limitations....50
5.10 Overall Discussion....51
5.11 Summary....52
LIVE MONITORING TOOL AND DEMONSTRATION....53
6.1 Overview....53
6.2 Live Monitoring Dashboard....53
6.3 Telemetry Demonstration....54
6.4 Sample Live Monitoring Output....55
6.5 Summary....55
CONCLUSION AND FUTURE WORK....56
7.1 Conclusion....56
7.2 Limitations....57
7.3 Future Scope....58
7.4 Summary....59
REFERENCES....61
APPENDIX....63
9.1 Telemetry Variable Definitions....63
9.2 Operational Rule Conditions....64

9.3 Prototype Screenshots.....65
SOURCE CODE.....67
10.1 Overview of the Source Code.....67
10.2 Telemetry Processing.....67
10.3 Condition-Based Reasoning.....68
10.4 Event Latching Logic.....69
10.5 Dashboard and Visualization.....73

ABSTRACT

Autonomous robots improve their capabilities mostly via machine learning retraining and the collection of huge datasets. Still, they do not possess the ability to reason about failures, utilize experience, and adjust to changing conditions. The proposed Cognitive Evolution Engine (CEE) is a middleware designed to enhance the evolution of robot's knowledge through experience and reflective reasoning instead of continuous model retraining. This system operates at the junction of perception and decision-making, translating perceptual observations into operational knowledge with the help of seven components: Experience Interpreter, Reflective Reasoning Engine, Operational Knowledge Graph, Knowledge Evolution Engine, Intelligent Forgetting Module, Curiosity Engine, and Knowledge Transfer Engine. CEE is designed for mobile robots and works based on ROS 2. The ability to maintain constant knowledge acquisition, updating, and use of the knowledge allows CEE to avoid the need for continual retraining of its algorithms. Warehouse navigation demonstrates CEE's efficiency in learning from navigation failures, improving its knowledge base and applying it in future situations.

Keywords — Autonomous Robots, Physical AI, Cognitive Middleware, Reflective Reasoning, knowledge Graph, Lifelong Learning, Explainable AI, ROS 2.

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 3.1	System Architecture of Cognitive Evolution Engine (CEE)	22
Figure 3.2	Overall Workflow of the Cognitive Evolution Engine	24
Figure 3.3	Data Flow Between CEE Modules	26
Figure 4.1	Telemetry Data Collection from the Simulated Robot	30
Figure 4.2	Operational Knowledge History in CEE	37
Figure 4.3	Main Dashboard of the Cognitive Evolution Engine	38
Figure 4.4	Telemetry Visualization in CEE	39
Figure 5.1	Telemetry Visualization and Operational Event Display	48
Figure 9.1	Interactive Mobile-Robot Simulator	65
Figure 9.2	Live Telemetry Monitoring Dashboard	65
Figure 9.3	Condition-Based Event Detection	66
Figure 9.4	Complete CEE Prototype	66

LIST OF TABLES

Table No.	Title	Page No.
Table 1.1	Project Objectives	14
Table 3.1	Major Modules of the Cognitive Evolution Engine	25
Table 3.2	Hardware Requirements	27
Table 3.3	Software Requirements	28
Table 4.1	Telemetry Variables Used in CEE	30
Table 4.2	Condition-Based Reasoning Rules	33
Table 4.3	Operational Rule Mapping	34
Table 4.4	Functions of the Knowledge Repository and History	36
Table 4.5	Functional Testing Methodology	40
Table 4.6	Proposed Evaluation Metrics	41
Table 5.1	Telemetry Variables and Their Monitoring Purposes	44
Table 5.2	Configured Conditions for Event Detection	45
Table 5.3	Operational Rule Mapping Results	45
Table 5.4	Event-Latching Behaviour	46
Table 5.5	Knowledge-History Functions	47
Table 5.6	Functional Testing Checklist	49
Table 5.7	Limitations of the CEE Prototype	51
Table 9.1	Telemetry Variable Definitions	63
Table 9.2	Operational Rule Conditions	64

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
CEE	Cognitive Evolution Engine
CPU	Central Processing Unit
DSR	Duplicate-Suppression Rate
EDA	Event-Detection Accuracy
ESP32	Espressif Systems 32-bit Microcontroller
ID	Identifier
IMU	Inertial Measurement Unit
JSON	JavaScript Object Notation
KG	Knowledge Graph
KRR	Knowledge-Retention Rate
LLM	Large Language Model
MQTT	Message Queuing Telemetry Transport
RAM	Random Access Memory
RGR	Rule-Generation Rate
RL	Reinforcement Learning
ROS 2	Robot Operating System 2
RPM	Revolutions Per Minute
TPS	Telemetry Processing Rate
VLA	Vision-Language-Action
VLM	Vision-Language Model

CHAPTER 1
INTRODUCTION

1.1 General

Autonomous robots are now used in logistics, manufacturing, inspection, health care, and service applications, where they must operate in environments that change from one day to the next. A robot that works in such a setting cannot rely only on what it was trained to do; it must also adapt to new conditions and take advantage of what it has already experienced. For this reason, continual and lifelong learning are widely regarded as important routes towards robotic autonomy [1], [2]. Cognitive robotic systems add to this view by placing memory, learning, reasoning, and knowledge retention at the centre of long-term autonomous operation [3], [4].

In a typical robotic pipeline, however, sensor measurements are used to make a prediction or trigger a reaction, and the experience behind that decision is then discarded. The robot does not keep a record of which conditions caused a problem, what the problem was interpreted to be, or what response was associated with it. Improving the robot therefore usually means collecting more data and retraining a model, which is expensive and slow, and which gives the robot no way to reflect on a failure while it is still operating.

This project addresses that gap with the Cognitive Evolution Engine (CEE), a middleware layer placed between perception and decision-making. CEE observes the robot's sensory data, identifies significant events, generates an operational rule for each event, and retains that rule so that it is available later. The report describes the architecture of CEE, the working prototype built to demonstrate it in simulation, and the functional tests used to examine its behaviour.

1.2 Problem Description

The problem addressed by this project is how a robot's operational experience, expressed as telemetry and the events detected in it, can be interpreted and stored as structured, reusable operational knowledge, without relying on continual retraining of a learning model. The solution should sit between the perception layer and the decision-making layer, should produce a clear link between each observation and the knowledge derived from it, and should be demonstrable without requiring a physical robot.

Several practical difficulties shape this problem. First, experience is normally not preserved: once a sensor reading has been used, nothing records that a particular combination of values corresponded to a particular event. Second, telemetry is continuous, so an anomaly that persists for many monitoring cycles can generate the same record again and again, making it hard to tell a new problem from an old one. Third, the relationship between telemetry and the resulting rule must be easy to inspect, which favours explicit, traceable reasoning over opaque models. Finally, there is no standard labelled dataset for this kind of experience-to-knowledge conversion, so the prototype has to be assessed through functional tests and clearly defined metrics rather than through conventional machine-learning scores.

1.3 Existing System

Existing research has produced important building blocks for long-term robotic intelligence. Continual and lifelong learning methods allow a robot to acquire new skills while retaining earlier ones [5], [6]. Persistent memory architectures combine spatial, episodic, and semantic memory for long-term operation [4], [3]. Knowledge graphs and retrieval-based systems provide structured knowledge for planning [7], [8]. Reflection-based methods evaluate predicted outcomes and revise plans [9], [10], and exploration and transfer methods help a robot to set its own learning goals and reuse what it has learnt [11], [12].

These approaches are usually studied separately or inside systems built for particular tasks. Most of them operate at the level of models, policies, or skills, and much of the knowledge they use comes from existing documents, demonstrations, or query-specific information, rather than from the robot's own operating experience. The integration of experience into a single, flexible cognitive layer remains an open topic.

At the practical level, conventional robot monitoring tools collect telemetry such as wheel speed, acceleration, inclination, distance, and temperature, and either display the current values or raise an alert when a threshold is crossed. They typically keep no structured history of the conditions that were detected, may report the same ongoing event repeatedly, and implement monitoring, reasoning, and record keeping as independent components.

1.4 Proposed System

The proposed Cognitive Evolution Engine is a middleware framework that connects robot telemetry with explicit reasoning and a structured operational knowledge history. In its

complete form, CEE is conceived as seven cooperating components: the Experience Interpreter, which turns perceptual observations into meaningful experience; the Reflective Reasoning Engine, which analyses the context, cause, and consequence of events; the Operational Knowledge Graph, which stores operational knowledge; the Knowledge Evolution Engine, which updates that knowledge; the Intelligent Forgetting Module, which removes knowledge that is no longer useful; the Curiosity Engine, which directs exploration; and the Knowledge Transfer Engine, which shares knowledge between robots. The framework is intended for mobile robots and for integration with ROS 2-based platforms.

The prototype developed in this project implements the first stage of this concept as a working software pipeline: Telemetry, State Interpretation, Event Detection, Rule Generation, and Knowledge Retention. An interactive mobile-robot simulator generates telemetry, which is transferred to CEE through MQTT. A deterministic, condition-based reasoning module evaluates the telemetry against predefined rules, maps each detected event to an operational rule, and records it in an in-session knowledge history. Event latching ensures that a continuously active anomaly produces a single entry, and a dashboard displays telemetry and recorded events in real time.

The reasoning in the prototype is deterministic rather than neural or reinforcement-learning-based, so every operational rule can be traced directly to the telemetry values that produced it. The remaining components of the full CEE concept, such as the Operational Knowledge Graph, Intelligent Forgetting Module, Curiosity Engine, and Knowledge Transfer Engine, are not implemented in the prototype and are listed as future work.

1.5 Objectives

The objectives of this project are summarised in Table 1.1.

No.	Objective
1	Design a cognitive middleware architecture that sits between robot perception and decision-making.
2	Build an interactive mobile-robot simulation that generates five telemetry variables: wheel RPM, IMU acceleration, pitch angle, ultrasonic distance, and system temperature.
3	Transfer the telemetry from the simulator to the CEE processing component through MQTT.
4	Implement deterministic, condition-based reasoning that interprets the telemetry and detects predefined operational events.
5	Map each detected event to an operational rule and record it in an in-session

	operational knowledge history.
6	Implement event latching so that a continuously active anomaly does not produce duplicate history entries.
7	Provide a real-time dashboard for telemetry, detected events, and rule history, and define functional tests and metrics for evaluating the pipeline.

Table 1.1: Project Objectives

1.6 Scope of the Project

The project covers a simulation-based prototype of the CEE middleware. It uses an interactive mobile-robot simulator, five telemetry variables, eight predefined condition-based rules, MQTT telemetry communication, an in-session operational knowledge history, an event-latching mechanism, and a real-time dashboard. Evaluation is limited to functional testing of this workflow in simulation.

The project does not implement neural-network or reinforcement-learning models, autonomous discovery of new rules, a formal semantic knowledge graph, or durable storage of knowledge across application restarts. It does not execute the intended responses (such as braking or torque adjustment) on physical hardware, and it does not claim validated detection accuracy on a real robot. The event labels produced by the system indicate that configured conditions were satisfied; they do not prove that the corresponding physical fault has occurred.

1.7 Contributions

The main contributions of this project are:

- **Cognitive middleware architecture:** a layer introduced between the robotic perception layer and the decision-making layer, which interprets experience and organises it as operational knowledge.
- **Knowledge-oriented rule generation:** a deterministic, rule-based cognitive process that converts interpreted robot states into operational rules, with a clear and inspectable link between telemetry and rule.
- **Event latching:** a state-tracking mechanism that records an event only when a condition becomes newly active, which keeps the operational history free of duplicate entries.
- **Operational knowledge history:** a record of detected events and their associated rules that can be inspected during the active session.

- **Working prototype:** an implementation using an interactive mobile-robot simulation, MQTT telemetry transmission, a reasoning module, a knowledge history, and a real-time visualisation dashboard.
- **Evaluation framework:** a set of functional tests and proposed metrics for event-detection accuracy, rule generation, knowledge retention, duplicate suppression, response latency, and telemetry processing rate.

1.8 Modules

The prototype is organised into five modules, which are listed with their responsibilities in Table 3.1 and implemented in Chapter 4.

- **Robot simulation:** generates the simulated robot's telemetry and provides an environment in which operating conditions can be observed.
- **Telemetry communication:** transfers telemetry from the simulator to the CEE through an MQTT publisher and broker.
- **Condition-based reasoning and rule mapping:** evaluates the telemetry against predefined conditions and associates each detected event with an operational rule.
- **Event latching and knowledge history:** suppresses duplicate entries and maintains the in-session record of detected events and rules.
- **Dashboard and visualisation:** displays telemetry, detected events, and the operational history.

1.9 Summary

This chapter explained why experience-driven adaptation matters for autonomous robots and why current systems do not retain and reuse operational experience. It introduced the Cognitive Evolution Engine as a middleware layer between perception and decision-making, described the scope of the simulation-based prototype, and set out the objectives, contributions, and modules of the project. The next chapter reviews the literature on which the project builds.

CHAPTER 2
LITERATURE REVIEW

2.1 Continual and Lifelong Learning in Robotics

Lesort et al. [1] proposed a framework for continual learning in robotics, drawing attention to changing data distributions, catastrophic forgetting, limits on storage and computation, and the practical needs of real robots. Romero et al. [2] studied lifelong open-ended learning from the viewpoint of cognitive architectures, and identified contextual long-term memory, learning methods, motivation, attention, and knowledge retrieval as important elements of long-term autonomous existence.

Later studies addressed continual learning in physical robots. Ayub et al. [13] combined continual learning, interactive machine learning, semantic reasoning, object recognition, and short- and long-term memory in one architecture for home-service robots. Long et al. [14] proposed DRAE, which brings together dynamic specialist selection, retrieval-augmented generation, hierarchical reinforcement learning (RL), and long-term knowledge accumulation. Meng et al. [5] studied lifelong RL that preserves and combines knowledge across different manipulation tasks, and Murillo-González and Liu [15] introduced action flow matching for continual robot learning. CLARE [6] achieves lightweight continual adaptation of vision-language-action (VLA) models while addressing catastrophic forgetting and the cost of updating the model.

These methods adapt and reuse knowledge effectively, but they mainly work at the level of models, policies, or skills. They do not record why a particular event occurred or what operational rule should follow from it.

2.2 Persistent Memory and Cognitive Architectures

Persistent memory is another important area for long-lasting robotic autonomy. Lei et al. [4] proposed RoboMemory, which combines spatial, temporal, episodic, and semantic memory with closed-loop planning and a dynamic knowledge graph (KG) for lifelong learning. Beetz et al. [3] introduced CRAM 2.0, a cognitive architecture that integrates semantic and episodic memory, generalised action plans, perception, constraint-based motion planning, and metacognitive reasoning. Zhang et al. [16] used multi-modal and historical knowledge graphs to support continual robot navigation.

These systems show how valuable it is to keep memory of earlier interactions. However, storing memory does not by itself turn the physical phenomena observed earlier into operational knowledge that the robot can apply directly to later decisions.

2.3 Knowledge Representation and Retrieval

Knowledge representation and retrieval have been investigated to support planning and execution. Vorobiov et al. [7] created ARRC, a knowledge-based and retrieval-augmented robotic system that uses motion primitives, action templates, safety rules, and stored information to structure action planning. Shek et al. [8] proposed KGLAMP, which merges relationship, property, and reachability graphs with language-model-based planning for adaptive multi-robot planning and replanning. Kümpel et al. [17] studied actionable knowledge graphs that connect natural-language instructions with reusable robotic actions, and Mon-Williams et al. [18] showed that embodied large language models (LLMs) can enable robots to complete complex tasks in unpredictable environments.

Together, these studies confirm that structured knowledge is necessary for robotic reasoning and action selection. Most of the knowledge they use, however, originates from material that already exists, from demonstrations, or from information tied to a specific query, and not from knowledge that evolves out of the robot's own experience.

2.4 Reflection and Closed-Loop Reasoning

Reflection and closed-loop reasoning have been widely discussed as means of improving robotic adaptation. Lou et al. [10] created ExploreVLM, which combines exploration, object-centric spatial representation, self-reflection, and execution verification. Feng et al. [9] proposed Reflective Planning, a technique in which a vision-language model (VLM) evaluates predicted future states and revises actions in long-horizon manipulation tasks. Sun et al. [19] combined retrieval-augmented hierarchical in-context RL with hindsight modular reflections, allowing earlier experience to be reused in later tasks.

These techniques demonstrate that planning and adaptation can be improved through reflection and experience-based evaluation. They motivate the reflective reasoning component of the CEE concept.

2.5 Autonomous Exploration and Knowledge Transfer

Autonomous exploration and knowledge transfer complement each other in lifelong learning. Fallas Hernández et al. [11] introduced a method for automatic sub-goal

generation that uses intrinsic motivation, bottom-up learning, and the linking of targets to set effective learning goals. Gao et al. [20] presented Arcadia, which merges exploration, grounding, scene reconstruction, shared embodied representations, and simulation-from-real learning into one framework for the full lifecycle of embodied lifelong learning. Yao et al. [12] proposed Primitive Prompt Learning, an efficient method of knowledge transfer based on motion-aware prompts and text.

These works highlight the importance of both gaining new experience and reusing what is already known, which correspond to the Curiosity Engine and Knowledge Transfer Engine in the CEE concept.

2.6 Existing Systems and Limitations

Previous studies have demonstrated several important features of robotic autonomy: continual adaptation, persistent memory, structured knowledge, introspection, exploration, and knowledge transfer. The literature nevertheless leaves practical gaps. Continual-learning methods focus on models and policies, memory systems store interactions without converting them into operational rules, knowledge-based systems draw mostly on prepared knowledge, and reflective methods are usually tied to a particular planning task.

Conventional monitoring tools, in turn, report current telemetry values or threshold alerts but keep no structured history, may repeat the same event on every monitoring cycle, and treat monitoring, reasoning, and record keeping as separate parts. None of these approaches provides a lightweight layer that interprets a robot's own telemetry, produces an operational rule for each significant event, and retains that rule in an organised form.

2.7 Research Gap

Based on this review, the gap addressed by this project is the absence of a comprehensive cognitive middleware framework that organises how a robot's experiences are understood and used. The project addresses this gap by combining, in one layer between perception and decision-making, telemetry interpretation, explicit event detection, operational rule generation, duplicate-free event recording, and an inspectable knowledge history, and by demonstrating this workflow in a working simulation-based prototype. The broader elements of the CEE concept, namely semantic knowledge networks, intelligent forgetting, curiosity-driven exploration, and knowledge sharing between robots, remain as future extensions.

2.8 Summary

This chapter reviewed work on continual and lifelong learning, persistent memory and cognitive architectures, knowledge representation, reflective reasoning, and autonomous exploration and knowledge transfer in robotics. These studies provide essential capabilities, but they are usually analysed separately or within task-specific systems, and few of them convert a robot's own operating experience into operational knowledge. This gap defines the research problem addressed by the project. The next chapter presents the system design.

CHAPTER 3
SYSTEM DESIGN

3.1 Existing System

In conventional robotic systems, telemetry monitoring, condition detection, and operational history are often handled by separate components. Sensor readings such as wheel speed, acceleration, inclination, distance, and system temperature are collected to monitor the robot's operating condition. However, the monitoring component may only display the current values or generate alerts when predefined thresholds are exceeded.

When an abnormal condition occurs, developers may need to inspect telemetry logs and identify the corresponding event manually. If the same abnormal condition continues for several monitoring cycles, the system may repeatedly report the same event, making it difficult to distinguish a newly occurring problem from an existing one. In addition, operational observations may not be organised into a structured history that can be reviewed during later analysis.

Traditional condition-based systems generally rely on predefined rules. Although these rules are useful for identifying known operating conditions, they do not automatically discover new rules or learn new behaviours from previous experiences. Monitoring, reasoning, and historical record keeping may also be implemented independently, increasing the complexity of maintaining and extending the system.

Therefore, there is a need for a modular middleware framework that integrates simulated robot telemetry, condition-based reasoning, operational rule recording, event duplication suppression, and visual monitoring within a single workflow. The Cognitive Evolution Engine (CEE) addresses this need through a software-based prototype that interprets telemetry conditions and records the corresponding operational rules for inspection.

3.2 Proposed System

The proposed system introduces the Cognitive Evolution Engine (CEE), a middleware framework designed to connect robot telemetry with condition-based reasoning and operational knowledge history. The prototype receives telemetry from an interactive mobile-robot simulator through the MQTT communication protocol. The received data is processed by the CEE reasoning component, which evaluates predefined conditions and identifies potential operational events.

The system monitors five principal telemetry variables: wheel RPM, IMU acceleration, pitch angle, ultrasonic distance, and system temperature. These variables are evaluated against predefined thresholds and combinations of conditions. When a rule is activated, the system interprets the corresponding event and records an operational rule in the in-session knowledge history.

The proposed system also incorporates event latching to prevent the same continuously active anomaly from generating duplicate history entries. A new entry is created when a condition changes from inactive to active, rather than on every subsequent evaluation while the condition remains active. This mechanism improves the readability of the operational history without being treated as a learning algorithm.

The architecture consists of five major functional stages:

1. **Robot simulation:** Generates the robot's telemetry values and allows its operating conditions to be observed.
2. **Telemetry communication:** Transfers telemetry from the simulator to the CEE through MQTT.
3. **Condition-based reasoning:** Evaluates incoming telemetry using predefined rules and thresholds.
4. **Operational knowledge history:** Records the interpreted events and their associated rules during the active session.
5. **Dashboard and visualization:** Displays telemetry values, monitoring information, and recorded operational events.

The prototype demonstrates experience-oriented operational record keeping and rule-based event interpretation in a simulated environment. It does not implement autonomous rule discovery, neural-network learning, or validated physical robot control. The knowledge history is maintained during the active session and should not be interpreted as durable storage across application restarts.

3.3 System Architecture

Figure 3.1 illustrates the overall architecture of the Cognitive Evolution Engine. The architecture can be understood through five functional layers: the simulation layer, communication layer, reasoning layer, knowledge-history layer, and presentation layer.

The simulation layer represents the mobile robot and generates telemetry values associated with its operating state. These values include wheel RPM, IMU acceleration, pitch angle, ultrasonic distance, and system temperature.

The communication layer uses MQTT to transfer telemetry messages from the simulator to the CEE processing component. This separation allows the telemetry producer and the reasoning component to communicate without being implemented as a single tightly coupled module.

The reasoning layer receives the telemetry values and evaluates them against predefined condition-based rules. For example, a high system temperature may indicate thermal overload, while a short ultrasonic distance combined with high wheel RPM may indicate that an obstacle is close to the robot. The reasoning component maps detected conditions to predefined operational interpretations.

The knowledge-history layer records the detected events and their associated operational rules. Event latching suppresses repeated entries while the same condition remains continuously active. The resulting history can be examined to understand which conditions were detected during the current session.

The presentation layer provides the dashboard and telemetry visualizations. It displays the available telemetry information and operational events, allowing the user to observe the relationship between robot state and the reasoning system's output.

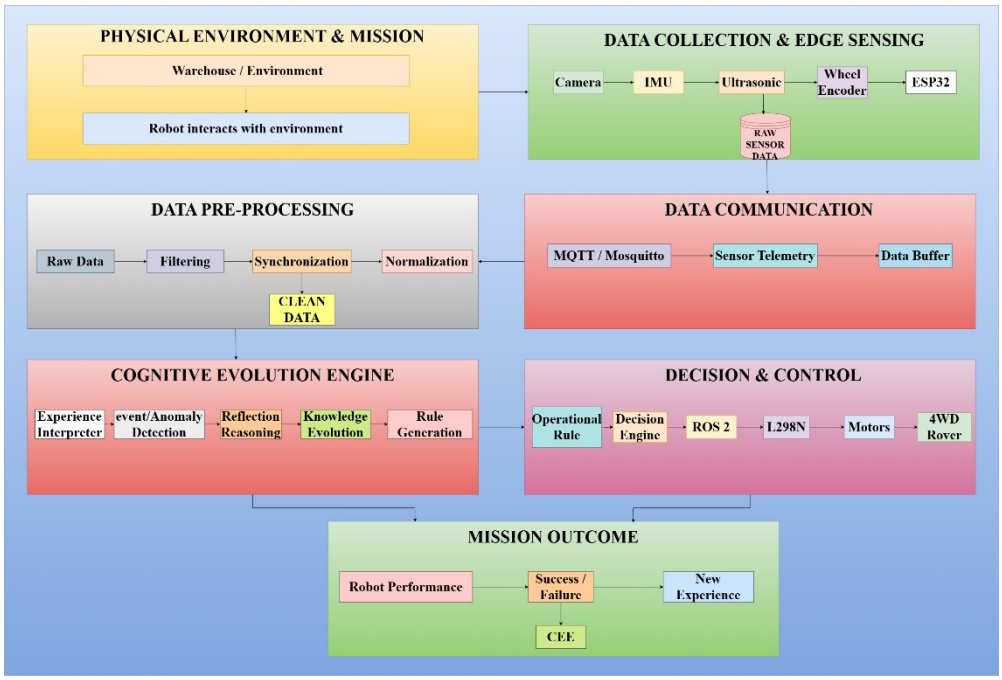


Figure 3.1: System Architecture of Cognitive Evolution Engine (CEE)

The architecture separates telemetry generation, communication, reasoning, history recording, and visualization into distinct functional responsibilities. This modular arrangement makes the prototype easier to understand, test, and extend. However, the current implementation should be considered a simulation-based proof of concept rather than a complete autonomous robotic control system.

3.4 Overall Workflow

The overall workflow of the Cognitive Evolution Engine proceeds through the following stages:

1. The interactive mobile-robot simulator generates telemetry values representing the robot's operating condition.
2. The telemetry values, including wheel RPM, IMU acceleration, pitch angle, ultrasonic distance, and system temperature, are published through MQTT.
3. The CEE receives the telemetry messages and extracts the values required for processing.
4. The reasoning component evaluates the incoming values against predefined thresholds and combinations of conditions.
5. When a condition is satisfied, the system identifies the corresponding operational event, such as thermal overload, obstacle proximity, possible wheel slip, or severe inclination.
6. The event is mapped to its predefined operational rule and prepared for recording in the knowledge history.
7. Event-latching logic checks whether the same condition is already active. If the condition has newly become active, the system records the event; if it remains active, duplicate entries are suppressed.
8. The dashboard displays the current telemetry information and available event history, enabling the user to monitor the simulated robot's operating state.

This workflow connects simulated telemetry with condition-based interpretation and structured operational history. The rule evaluation depends on explicitly defined conditions, while the history provides a record of the events identified during the session. The system does not automatically create new reasoning rules from the history.

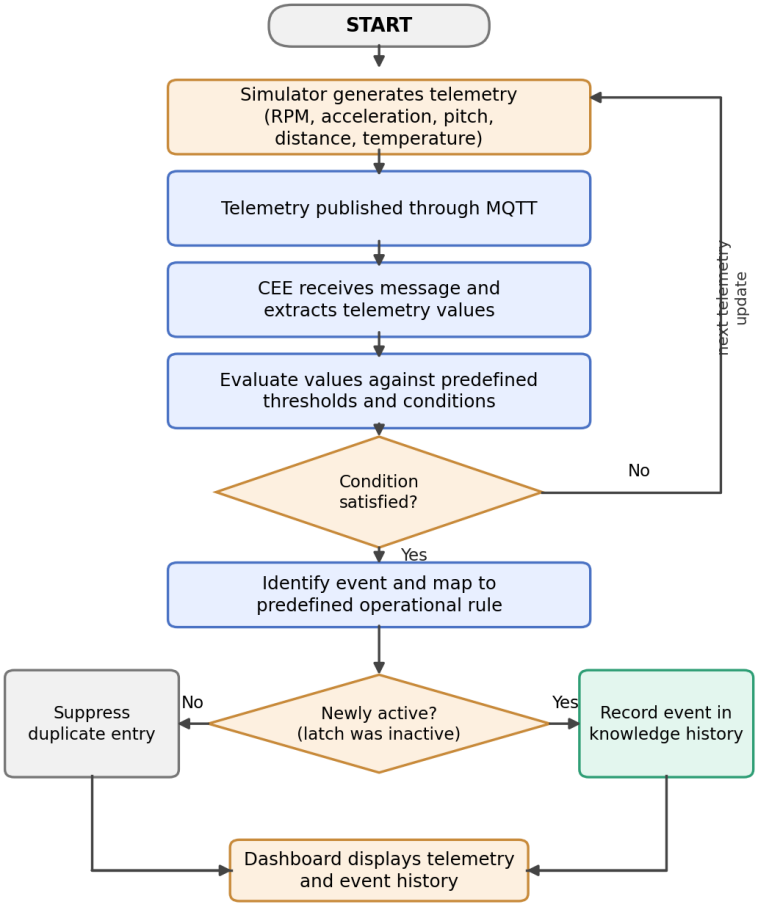


Figure 3.2: Overall Workflow of the Cognitive Evolution Engine

3.5 System Modules

Table 3.1 describes the major functional modules of the CEE prototype and their responsibilities.

Module	Responsibility
Robot Simulation	Generates simulated mobile-robot telemetry and provides an interactive environment for observing operating conditions.
Telemetry Publisher	Publishes robot telemetry values through the MQTT communication protocol.
MQTT Communication	Transfers telemetry messages between the simulator and the CEE processing component.
Telemetry Processing	Receives and extracts telemetry values required by the reasoning component.
Condition-Based Reasoning	Evaluates telemetry against predefined thresholds and combinations of conditions.
Operational Rule Mapping	Associates detected conditions with predefined event descriptions and operational rules.

Module	Responsibility
Event Latching	Suppresses duplicate entries while an anomaly remains continuously active.
Knowledge History	Maintains an in-session record of detected events and associated operational rules.
Dashboard and Visualization	Displays telemetry values, monitoring information, and the operational event history.

Table 3.1: Major Modules of the Cognitive Evolution Engine

The modules are designed to provide a clear separation between telemetry collection, communication, event interpretation, and visualization. The reasoning component determines whether a predefined condition is satisfied, while the event-latching module manages repeated detections of an ongoing condition. The knowledge-history module records the results for inspection, and the dashboard presents the information to the user.

The module structure also provides a foundation for future extensions, such as persistent history storage, more advanced reasoning mechanisms, and integration with physical robot platforms. These extensions are not considered implemented features of the current prototype.

3.6 Data Flow

Figure 3.3 illustrates the flow of telemetry and operational information through the CEE system. The data flow begins in the robot simulator and continues through MQTT communication, telemetry processing, condition-based reasoning, event recording, and dashboard visualization.

Initially, the simulator generates values for the selected telemetry variables. These values are transmitted to the CEE through MQTT messages. After receiving a message, the telemetry-processing component extracts the required values and provides them to the reasoning component.

The reasoning component evaluates the telemetry against the predefined conditions. If a condition is satisfied, the system determines the associated event and operational rule. The event-latching mechanism then checks whether the condition was already active. A newly activated condition can produce a history entry, whereas a continuously active condition does not produce repeated entries.

The resulting operational information is added to the in-session knowledge history and made available to the dashboard. The dashboard displays the telemetry and recorded events, allowing the user to examine the current state and the conditions detected during the session.

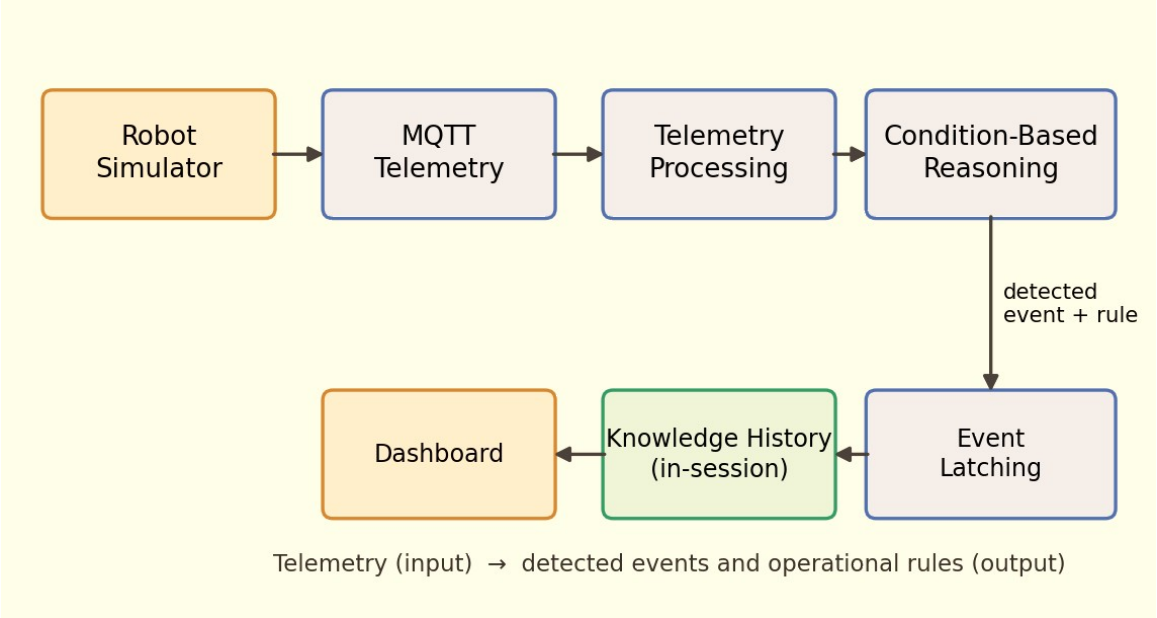


Figure 3.3: Data Flow Between CEE Modules

The primary data flow can be summarised as:

Robot Simulator → MQTT Telemetry → Telemetry Processing → Condition-Based Reasoning → Event Latching → Knowledge History → Dashboard

The telemetry values form the input to the reasoning process, while the detected events and associated operational rules form the main outputs. The current knowledge history is maintained in the active application session; durable database storage across restarts is not established by the implemented prototype.

3.7 Hardware Requirements

Table 3.2 presents the hardware requirements for developing and running the CEE prototype. Since the current implementation uses an interactive robot simulator, a physical robot is not required for the demonstrated workflow.

Component	Specification / Purpose
Development Computer	A computer capable of running the robot simulator, MQTT communication components, reasoning logic, and dashboard.

Component	Specification / Purpose
Processor (CPU)	Executes the simulation, telemetry processing, rule evaluation, and dashboard operations.
Memory (RAM)	Provides memory for the application, simulator, telemetry processing, and visualization components.
Storage	Stores the project source code, configuration files, and any application files required by the prototype.
Network Connectivity	Supports MQTT communication between the simulator and the CEE components, depending on the deployment configuration.
Physical Robot	Not required for the current simulation-based demonstration.

Table 3.2: Hardware Requirements

The hardware requirements depend on the computer used to run the simulator and the associated software components. The prototype focuses on the software workflow of receiving telemetry, evaluating predefined conditions, recording operational events, and displaying the results.

A physical robot with sensors and motor-control interfaces would be required for experimental validation in a real environment. Such validation would need to address sensor reliability, communication delays, actuator safety, and the behaviour of the reasoning system under real operating conditions. The current simulation does not establish those physical capabilities.

3.8 Software Requirements

Table 3.3 lists the principal software components and their purposes in the CEE prototype. Exact versions and environment-specific dependencies depend on the configuration used in the implementation.

Component	Purpose
Python	Implements the telemetry-processing logic, condition-based reasoning, event handling, and supporting application functionality where used by the project.
Robot Simulation Software	Provides the interactive simulated mobile robot and generates telemetry values.
MQTT Broker	Facilitates message exchange between the telemetry publisher and the CEE subscriber.
MQTT Client Library	Publishes and receives telemetry messages through MQTT.
Dashboard Framework	Provides the interface for displaying telemetry information and operational event history.
Data Visualization Library	Displays telemetry plots and other visual monitoring elements where included in the implementation.

Development Environment	Supports code development, execution, debugging, and testing.
Operating System	Hosts the simulator, communication components, reasoning application, and dashboard.

Table 3.3: Software Requirements

Python-based processing, MQTT communication, and dashboard visualization provide the main software foundation of the CEE prototype. The MQTT broker and client components support telemetry transfer, while the reasoning logic evaluates incoming values using predefined conditions.

The dashboard and visualization components present telemetry and event history in a form that can be inspected during testing. Exact package versions and operating-system details depend on the environment used for the demonstration and are not assumed in this report.

3.9 Summary

This chapter described the system design of the Cognitive Evolution Engine, including the existing system, proposed system, architecture, workflow, modules, data flow, and hardware and software requirements. The proposed design connects an interactive mobile-robot simulator to the CEE through MQTT and evaluates the resulting telemetry using predefined condition-based rules.

Detected conditions are associated with operational rules and recorded in an in-session knowledge history. Event latching suppresses duplicate records while the same anomaly remains continuously active, and the dashboard presents telemetry information and detected events for monitoring.

The modular design separates telemetry communication, reasoning, history management, and visualization. The current implementation demonstrates a simulation-based framework for condition interpretation and operational event recording; it does not establish autonomous rule learning, durable knowledge persistence across restarts, or validated physical robot control. The next chapter explains the implementation and methodology used to develop and test these components.

CHAPTER 4
IMPLEMENTATION AND METHODOLOGY

4.1 Mobile Robot Simulation

The CEE prototype uses an interactive mobile-robot simulator to generate telemetry representing the operating condition of a robot. The simulator provides a controlled environment in which different operating states can be observed without requiring a physical robot. This approach makes it possible to demonstrate telemetry monitoring and condition-based reasoning while reducing the hardware requirements of development and testing.

The simulated robot produces five principal telemetry variables: wheel revolutions per minute (RPM), IMU acceleration, pitch angle, ultrasonic distance, and system temperature. These variables represent different aspects of the robot's movement, orientation, surrounding environment, and operating condition.

The simulator provides the input data required by the CEE reasoning component. As the telemetry values change, the system evaluates whether any predefined conditions have been satisfied. For example, a short ultrasonic distance combined with high wheel RPM may indicate obstacle proximity, while a pitch angle exceeding the specified threshold may indicate severe inclination.

The simulation is used to demonstrate the software workflow from telemetry generation to event interpretation and recording. The results should therefore be understood as evidence of the prototype's operation in a simulated environment, not as validation of the same behaviour on a physical robot.

4.2 Telemetry Data Collection

Telemetry data collection is the process of obtaining the robot's operating values from the simulation and making them available to the Cognitive Evolution Engine. The collected data provides the input for condition-based reasoning and enables the system to monitor changes in the simulated robot's state.

The prototype monitors five telemetry variables, as described in Table 4.1.

Telemetry variable	Description	Purpose
Wheel RPM	Rotational speed of the robot's wheels	Helps identify high wheel speed and possible movement-related issues.
IMU Acceleration	Acceleration value obtained from the simulated inertial measurement unit	Supports the interpretation of movement and possible collision or traction conditions.
Pitch Angle	Forward or backward inclination of the robot	Helps detect severe inclination.
Ultrasonic Distance	Estimated distance to an obstacle	Supports obstacle proximity detection and identification of invalid or unusually large readings.
System Temperature	Temperature value associated with the system	Helps identify possible thermal overload.

Table 4.1: Telemetry Variables Used in CEE

The collected telemetry values are represented by the state vector:

$$\mathbf{x}_t = [R_t,\ A_t,\ P_t,\ U_t,\ T_t]^T$$

where R_t represents wheel RPM, A_t represents IMU acceleration, P_t represents pitch angle, U_t represents ultrasonic distance, and T_t represents system temperature at time t.

The telemetry values are passed to the reasoning component for evaluation. When the values satisfy one or more predefined conditions, the system identifies the corresponding operational event. The quality of the reasoning output therefore depends on the values received and the correctness of the defined thresholds.

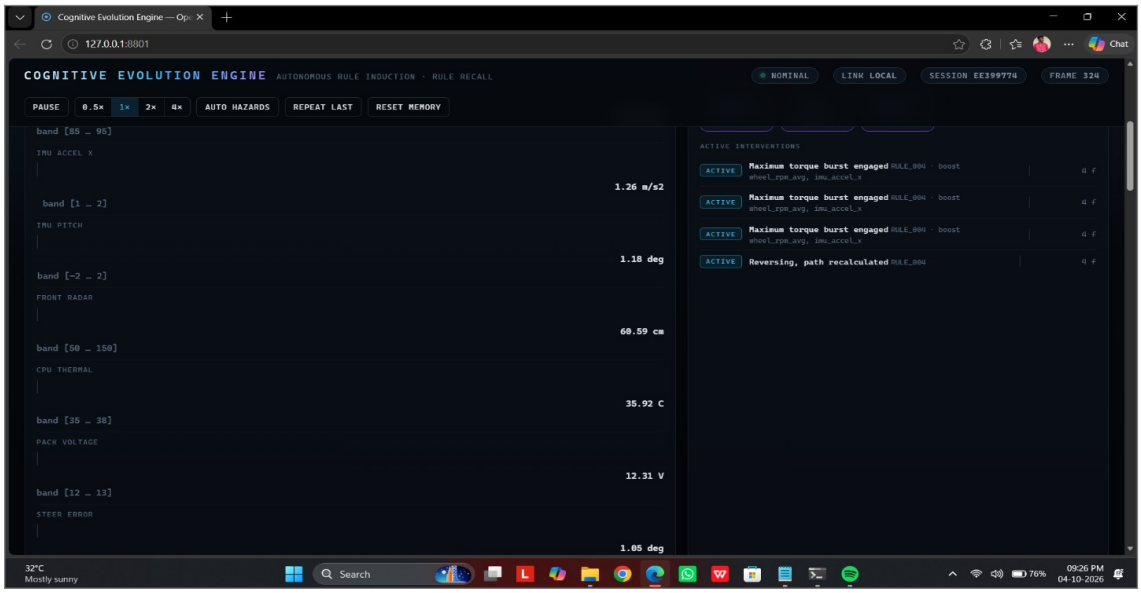


Figure 4.1: Telemetry Data Collection from the Simulated Robot

4.3 MQTT-Based Communication

The CEE prototype uses the Message Queuing Telemetry Transport (MQTT) protocol to transfer telemetry data between the mobile-robot simulator and the reasoning system. MQTT is a lightweight messaging protocol that supports communication between software components through a broker.

In the implemented workflow, the simulator acts as the source of telemetry information. The telemetry publisher sends messages to the MQTT broker, and the CEE component receives the messages through an appropriate subscription. The received values are then passed to the telemetry-processing and reasoning modules.

The communication process can be summarised as follows:

1. The simulator generates telemetry values.
2. The publisher prepares the values for transmission.
3. The MQTT client publishes the telemetry message to the configured broker and topic.
4. The CEE subscriber receives the message.
5. The received telemetry values are made available for condition evaluation.

This arrangement separates telemetry generation from reasoning. Consequently, the simulator and reasoning components can be developed and tested as separate functional units, provided that they use compatible message formats and communication settings.

MQTT communication depends on the availability and correct configuration of the broker, topic, network, and client settings. Connection failures can prevent telemetry from reaching the reasoning component. Such communication problems should be distinguished from errors in condition-based reasoning, because a message that is not received cannot be evaluated by the reasoning engine.

4.4 Telemetry Variable Processing

After telemetry messages are received, the system processes the values required by the reasoning component. This stage connects raw telemetry data with the condition-evaluation logic. Each variable is interpreted according to its role in the predefined operational conditions.

Wheel RPM is used to assess wheel rotation speed. IMU acceleration provides information about the robot's movement. Pitch angle indicates the robot's inclination, while ultrasonic distance provides information about the estimated distance to nearby obstacles. System temperature is monitored to identify conditions that may indicate excessive heat.

The values are evaluated individually or in combination, depending on the rule being checked. For example, a high wheel RPM alone does not necessarily establish wheel slip. The prototype uses a combination of wheel RPM and acceleration thresholds to flag a possible wheel-slip condition.

Similarly, a low ultrasonic distance combined with high wheel RPM is used to identify obstacle proximity. Combining multiple telemetry variables allows the system to represent conditions more specifically than relying on a single measurement.

The processing stage supplies the relevant values to the condition-based reasoning module. The system then evaluates the conditions using the predefined thresholds and logical operators specified for each rule.

The current prototype uses deterministic condition-based processing. It does not use a trained machine-learning model to classify the telemetry, and it does not autonomously learn new thresholds from previously recorded events.

4.5 Condition-Based Reasoning

Condition-based reasoning is the main event-interpretation component of the Cognitive Evolution Engine. It evaluates incoming telemetry values against predefined rules to identify operating conditions that may require attention.

Each rule contains a condition or combination of conditions and an associated event interpretation. When the condition is satisfied, the system identifies the corresponding event and passes it to the operational rule-mapping and history components.

Table 4.2 provides examples of the conditions implemented in the prototype.

No.	Condition	Event interpretation
1	All selected telemetry values are zero	Possible telemetry failure
2	System temperature >75°C	Thermal overload
3	Ultrasonic distance >900 cm	Possible sensor blindness or invalid reading

No.	Condition	Event interpretation
4	Ultrasonic distance <10 cm and IMU acceleration <0	Possible wall collision
5	Ultrasonic distance <25 cm and wheel RPM >50	Obstacle proximity
6	Wheel RPM >120 and IMU acceleration <0.8	Possible wheel slip
7	Wheel RPM <50, IMU acceleration <0.3, and ultrasonic distance >30 cm	Possible traction loss
8	Absolute pitch angle >15°	Severe inclination

Table 4.2: Condition-Based Reasoning Rules

When a rule's condition is satisfied, the reasoning component assigns the corresponding interpretation. For instance, if system temperature exceeds 75°C, the system identifies a thermal-overload condition. If the absolute pitch angle exceeds 15°, the system identifies severe inclination.

The conditions that combine several telemetry variables are evaluated using the specified logical relationships. This allows the prototype to interpret a combination of measurements rather than treating each value as an independent event.

The reasoning process can be expressed as:

$$E_t = f(\mathbf{x}_t, \mathcal{R})$$

where $\mathbf{x}_t$ is the current telemetry state, R is the set of predefined rules, and E_t represents the event or events identified at time t.

The function f represents deterministic condition evaluation, not a learned prediction model. The output is dependent on the predefined rules and the telemetry values received by the system.

The rules demonstrate how telemetry can be mapped to operational interpretations. However, the resulting labels are potential interpretations of simulated conditions; they do not independently prove that a physical collision, wheel slip, or traction loss has occurred.

4.6 Operational Rule Mapping

Operational rule mapping connects a detected telemetry condition with a predefined operational description or response rule. After the reasoning component identifies an event, the system associates it with the corresponding rule representation so that the event can be recorded and displayed.

For example, the telemetry-failure condition is associated with the rule:

```
IF data_stream_lost THEN trigger_hardware_safing_mode()
```

This rule expresses the intended operational response to a loss of telemetry. It should be understood as a rule representation in the prototype, rather than evidence that a physical hardware-safing action was executed on a real robot.

Other detected conditions are associated with their respective operational interpretations, as summarised in Table 4.3.

Detected condition	Operational interpretation or rule intent
Telemetry failure	Request a hardware-safing response.
Thermal overload	Throttle cognitive processing.
Invalid or excessively large ultrasonic reading	Use IMU-based interpretation as an alternative information source.
Possible wall collision	Reverse and recalculate the movement response.
Obstacle proximity	Request emergency braking.
Possible wheel slip	Cap motor RPM by 50%.
Possible traction loss	Increase torque response.
Severe inclination	Enable crawl mode.

Table 4.3: Operational Rule Mapping

The rule mapping stage provides a structured connection between detected events and their associated operational intentions. It also makes the recorded history easier to inspect because each event can be associated with a recognisable rule description.

The operational rule representations do not, by themselves, establish that the listed actions were physically carried out. Actual motor control, braking, torque adjustment, and hardware-safing behaviour would require an implemented and validated actuator-control interface. The current prototype demonstrates condition detection and rule representation in a simulation-based workflow.

4.7 Event Latching and Duplicate Suppression

A continuously active anomaly may satisfy the same condition during multiple consecutive telemetry evaluations. If every evaluation creates a new history entry, the operational history may contain many duplicate records for a single ongoing event.

To reduce this duplication, the CEE prototype implements event latching. The mechanism tracks whether a condition is already active and records a new event when the condition changes from inactive to active. While the condition remains active, subsequent evaluations do not create additional entries for that same ongoing event.

The basic behaviour is illustrated below.

1. The system receives telemetry and evaluates a predefined condition.
2. If the condition is not satisfied, the event remains inactive.
3. When the condition becomes satisfied, the event transitions from inactive to active and can be recorded.
4. While the condition remains active, duplicate entries are suppressed.
5. When the condition becomes inactive, the latch can be reset so a later activation can be recognised as a new event.

The event-latching behaviour can be represented as:

$$L_t = \begin{cases} 1, & \text{if the condition is active at } t \\ 0, & \text{otherwise} \end{cases}$$

A new event entry is generated on the inactive-to-active transition:

$$N_t = A_t \wedge \neg L_{t-1}$$

where A_t denotes whether the condition is active at time t, and L_{t-1} represents its previous active state. Here, $N_t = 1$ indicates that a new event entry should be created. This expression describes the intended transition logic; the implementation must maintain the previous state for each tracked condition.

Event latching improves the organisation of the operational history by reducing repeated entries for continuously active conditions. It is a state-tracking and duplicate-suppression mechanism, not an autonomous learning process. It does not independently identify new rules or determine whether a previously detected event has been resolved beyond the condition becoming inactive.

4.8 Knowledge Repository and History

The knowledge repository and history component maintains records of operational events detected by the reasoning engine. When a condition becomes active and passes the event-

latching check, the associated event and operational rule can be added to the in-session history.

This component provides a record that can be inspected through the dashboard. It helps the user review which conditions were detected and which rule descriptions were associated with them during the current session.

The repository is intended to organise operational observations in a consistent form. Rather than presenting only the latest telemetry values, the history allows detected events to remain available for review while the session is active.

Formally, each knowledge item can be represented as a triple consisting of the rule identifier, the detected event, and the associated operational rule:

$$K_i = (ID_i,\ E_i,\ R_i)$$

where ID_i is the identifier of the rule, E_i is the event that was detected, and R_i is the operational rule. When a new item is recorded, the knowledge history is updated by adding it to the existing set:

$$K_{t+1} = K_t \cup \{K_i\}$$

In the prototype this set exists only for the duration of the active session.

Table 4.4 summarises the role of the history component.

Function	Description
Event recording	Stores detected operational events in the active session.
Rule association	Connects an event with its predefined operational rule.
Duplicate suppression	Works with event latching to avoid repeated entries for an ongoing condition.
History display	Makes recorded operational events available for inspection through the dashboard.
Session-based retention	Maintains the history during the active application session.

Table 4.4: Functions of the Knowledge Repository and History

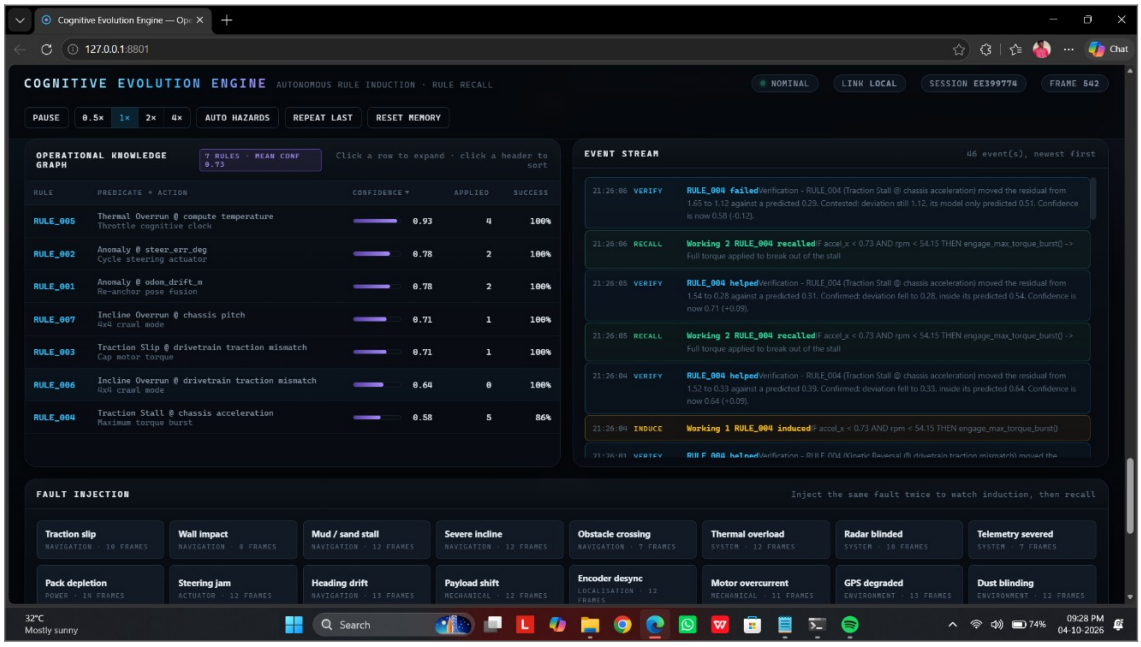


Figure 4.2: Operational Knowledge History in CEE

In the current implementation, the repository is an in-session operational history rather than a formal semantic knowledge graph or a durable database-backed knowledge store. Persistent storage across application restarts has not been established. The recorded events therefore demonstrate operational knowledge acquisition and retention within the active session, but not long-term knowledge persistence.

4.9 Dashboard Implementation

The CEE dashboard provides a visual interface for observing telemetry and reviewing detected operational events. It presents information produced by the simulator, communication components, and reasoning engine in a form that is easier to inspect than raw messages or console output alone.

The dashboard is intended to support monitoring of the robot's current simulated state and the events identified by the predefined rules. The telemetry display helps the user observe changes in the monitored variables, while the operational history presents the events recorded by the system.

The dashboard is connected to the prototype's processing workflow so that available telemetry and event information can be displayed during execution. Its principal functions include:

- Displaying telemetry values associated with the simulated robot.
- Presenting detected conditions and operational events.

- Showing the operational rule history maintained during the session.
- Supporting visual inspection of telemetry changes and event detection.
- Providing a convenient interface for demonstrating the CEE workflow.

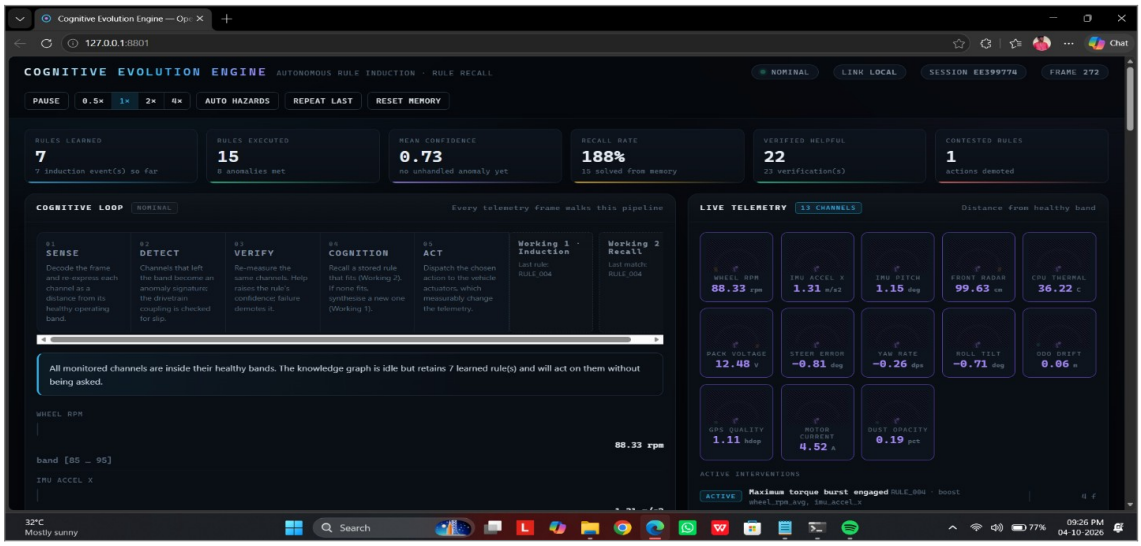


Figure 4.3: Main Dashboard of the Cognitive Evolution Engine

The dashboard serves as the presentation layer of the prototype. It makes the system's outputs visible to the user, but the displayed information does not independently validate the correctness of the reasoning rules. The accuracy of event interpretation still depends on the implemented conditions and the quality of the incoming telemetry.

4.10 Telemetry Visualization

Telemetry visualization provides a graphical representation of the values generated by the simulated robot. It complements the numerical telemetry display by helping the user observe changes and identify relationships between the monitored variables.

The prototype visualizes telemetry associated with wheel RPM, IMU acceleration, pitch angle, ultrasonic distance, and system temperature. These variables provide different perspectives on the simulated robot's movement and operating state.

For example, a change in wheel RPM can be examined alongside acceleration values. Ultrasonic distance can be observed when the simulated robot approaches an obstacle, while pitch angle can be reviewed when the robot enters an inclined state. System temperature can be monitored against the predefined thermal threshold.

The visualizations also help the user relate changes in telemetry to the events reported by the reasoning engine. When a value or combination of values satisfies a rule, the corresponding event can be compared with the visible telemetry history, where available.



Figure 4.4: Telemetry Visualization in CEE

Visualization improves the interpretability of the monitoring workflow. However, a graphical trend alone does not prove the cause of an anomaly. The event classification is determined by the predefined reasoning conditions, and the plots provide supporting context for examining those classifications.

4.11 Testing Methodology

The CEE prototype is tested in a simulated environment to verify the operation of telemetry communication, condition-based reasoning, operational rule mapping, event latching, knowledge history, and dashboard visualization.

The testing methodology focuses on functional verification: whether the components behave according to their implemented logic under the tested conditions. It does not establish physical robot performance, safety certification, or statistical detection accuracy across a representative real-world dataset.

The principal functional checks are summarised in Table 4.5.

Test area	Test procedure	Expected behaviour
Telemetry collection	Observe telemetry values generated by the simulator.	The selected telemetry values are available for processing.

Test area	Test procedure	Expected behaviour
MQTT communication	Run the configured publisher and subscriber workflow.	Telemetry messages are delivered to the CEE when the broker and clients are correctly configured.
Telemetry processing	Supply or observe values for the monitored variables.	The processing component makes the required values available to the reasoning logic.
Condition-based reasoning	Evaluate telemetry that satisfies a predefined rule.	The corresponding event interpretation is produced.
Operational rule mapping	Trigger a known condition.	The detected event is associated with its predefined rule.
Event latching	Keep a condition continuously active across repeated evaluations.	Duplicate entries for the same ongoing condition are suppressed.
Latch reset	Allow a condition to become inactive and activate it again.	The subsequent activation can be recorded as a new event.
Knowledge history	Trigger a condition and inspect the history.	The event is available in the in-session operational history.
Dashboard visualization	Observe the dashboard while telemetry is available.	The available telemetry and event information are displayed.

Table 4.5: Functional Testing Methodology

Testing should be performed by triggering each condition independently where possible, then observing whether the expected event and rule are produced. For combined conditions, the relevant telemetry values should be adjusted so that the complete logical expression is satisfied. This helps distinguish a correctly functioning rule from a condition that was not actually triggered.

For event-latching tests, the same condition should be kept active through several consecutive evaluations. The history should then be inspected to determine whether duplicate entries were suppressed. After the condition becomes inactive, it should be activated again to verify whether a new event can be recorded.

Communication tests should also consider the availability of the MQTT broker and the configuration of the publisher and subscriber. A connection failure should be recorded as a communication issue rather than automatically interpreted as a failure of the reasoning logic.

To support quantitative evaluation in future experiments, six metrics are defined for the deterministic CEE pipeline (Table 4.6). They are used in place of conventional machine-

learning indicators, which do not apply to a rule-based reasoning process. Values for these metrics are not reported in this report, because they have not been measured using a defined test procedure.

Metric	Definition	Terms
Event-detection accuracy (EDA)	$EDA = (N_c / N) \times 100$	N is the number of test events and N_c is the number of events detected correctly.
Rule-generation rate (RGR)	$RGR = (N_r / N_c) \times 100$	N_r is the number of correctly detected events for which the intended rule was generated.
Knowledge-retention rate (KRR)	$KRR = (N_k / N_r) \times 100$	N_k is the number of generated rules that were successfully retained.
Duplicate-suppression rate (DSR)	$DSR = (1 - N_d / N_r) \times 100$	N_d is the number of events for which a rule was generated more than once.
Cognitive response latency	$T_c = T_r - T_e$	T_e is the time the event was received and T_r is the time the rule was generated.
Telemetry processing rate (TPS)	$TPS = N_m / T$	N_m is the number of telemetry messages processed in the time interval T.

Table 4.6: Proposed Evaluation Metrics

The outcomes reported in this report reflect only the tests actually performed. Quantitative accuracy, latency, throughput, and reliability values are not reported, because they have not been measured using a defined test procedure.

4.12 Summary

This chapter described the implementation and methodology of the Cognitive Evolution Engine. It explained how the simulated mobile robot generates telemetry, how MQTT transfers the data to the CEE, and how the telemetry-processing component prepares the values for condition-based reasoning.

The chapter also described how predefined rules interpret telemetry conditions, how operational rule mapping associates detected events with rule descriptions, and how event latching suppresses duplicate entries for continuously active anomalies. The knowledge-history component maintains event records during the active session, while the dashboard and telemetry visualizations provide a graphical view of the system's operation.

Finally, the testing methodology outlined functional checks for telemetry collection, communication, reasoning, rule mapping, event latching, history recording, and visualization. The implemented system demonstrates a modular, simulation-based

approach to operational event interpretation and history management. Autonomous rule discovery, durable persistence across restarts, and validated physical robot actuation remain outside the scope of the current prototype.

The next chapter presents the results and discussion of the implemented CEE system, including the observed telemetry behaviour, event detection, operational rule recording, event-latching behaviour, and dashboard output.

CHAPTER 5
RESULTS AND DISCUSSION

5.1 Experimental Setup

The CEE prototype was evaluated in a software-based environment consisting of an interactive mobile-robot simulator, MQTT-based telemetry communication, a condition-based reasoning component, an in-session operational knowledge history, and a dashboard for monitoring and visualization.

The simulator generated telemetry values representing the operating state of the robot. The five principal variables were wheel RPM, IMU acceleration, pitch angle, ultrasonic distance, and system temperature. These values were transmitted to the CEE through MQTT and evaluated against predefined conditions.

The evaluation focused on the functional behaviour of the system rather than on training a machine-learning model. The main objectives were to observe whether telemetry could be received and processed, whether the predefined conditions could identify the intended events, whether the corresponding operational rules could be recorded, and whether event latching could suppress duplicate records for continuously active conditions.

The dashboard was used to observe telemetry information and review the recorded events during the active session. The experiments were conducted in simulation; therefore, the results demonstrate the operation of the software prototype and do not establish performance on a physical robot.

5.2 Telemetry Monitoring Results

Telemetry monitoring was used to examine whether the CEE prototype could receive and display the variables generated by the mobile-robot simulator. The monitored variables represented the robot's wheel rotation, acceleration, inclination, distance to nearby obstacles, and system temperature.

The telemetry values served as the primary inputs to the reasoning component. Observing these variables made it possible to relate changes in the simulated robot's state to the predefined conditions used by the system.

Table 5.1 summarises the role of each telemetry variable in the monitoring process.

Telemetry variable	Monitoring purpose	Related condition
Wheel RPM	Observes wheel rotational speed	Possible wheel slip and obstacle proximity
IMU acceleration	Observes simulated acceleration	Possible collision, wheel slip, and traction loss
Pitch angle	Observes robot inclination	Severe inclination
Ultrasonic distance	Observes estimated obstacle distance	Obstacle proximity, possible collision, and invalid reading
System temperature	Observes system temperature	Thermal overload

Table 5.1: Telemetry Variables and Their Monitoring Purposes

The monitoring stage provides the information required for the remaining stages of the CEE workflow. Telemetry monitoring alone does not determine whether an anomaly exists; the interpretation is performed by the condition-based reasoning component.

The results should be interpreted as evidence of the prototype's telemetry-monitoring functionality. Numerical ranges, update frequency, message-loss rate, and monitoring latency are not reported here, because they have not been measured from recorded execution logs.

5.3 Condition-Based Event Detection

The condition-based reasoning component evaluates telemetry values against predefined thresholds and combinations of conditions. When a condition is satisfied, the system associates the observed telemetry state with the corresponding operational event.

Table 5.2 presents the principal event types supported by the configured rules.

No.	Rule condition	Detected event
1	All selected telemetry values are zero	Possible telemetry failure
2	System temperature exceeds 75°C	Thermal overload
3	Ultrasonic distance exceeds 900 cm	Possible sensor blindness or invalid reading
4	Ultrasonic distance is below 10 cm and IMU acceleration is below zero	Possible wall collision
5	Ultrasonic distance is below 25 cm and wheel RPM exceeds 50	Obstacle proximity
6	Wheel RPM exceeds 120 and IMU acceleration is below 0.8	Possible wheel slip

No.	Rule condition	Detected event
7	Wheel RPM is below 50, IMU acceleration is below 0.3, and ultrasonic distance exceeds 30 cm	Possible traction loss
8	Absolute pitch angle exceeds 15°	Severe inclination

Table 5.2: Configured Conditions for Event Detection

The rules demonstrate how multiple telemetry variables can be used to identify predefined operating conditions. Some rules depend on a single threshold, such as system temperature or pitch angle. Others require multiple conditions to be satisfied simultaneously, such as the rule for obstacle proximity.

This design provides a straightforward and interpretable relationship between the incoming telemetry and the resulting event. The event labels, however, represent interpretations defined by the implemented rules. For example, the wheel-slip rule identifies a possible wheel-slip condition; it does not establish that physical wheel slip has occurred.

The demonstrated capability is deterministic event detection using predefined conditions. The prototype does not automatically discover new event categories or learn new thresholds from the recorded history.

5.4 Operational Rule Generation

After a condition is detected, the CEE maps the event to its associated operational rule. This provides a structured representation of the interpretation and intended response associated with the observed telemetry.

Table 5.3 provides examples of the operational rule mappings used in the prototype.

Detected event	Associated operational rule or intended response
Possible telemetry failure	IF data_stream_lost THEN trigger_hardware_safing_mode()
Thermal overload	Throttle cognitive processing
Possible sensor blindness	Use IMU-based interpretation
Possible wall collision	Reverse and recalculate
Obstacle proximity	Request emergency braking
Possible wheel slip	Cap motor RPM by 50%
Possible traction loss	Increase torque response
Severe inclination	Enable crawl mode

Table 5.3: Operational Rule Mapping Results

The mapping stage links an event interpretation to a recognisable rule description. These descriptions can then be recorded in the operational history and displayed through the dashboard.

This approach improves traceability because a recorded event can be examined alongside the condition and the associated operational response. It also makes the system easier to extend: additional predefined conditions and corresponding rule descriptions could be introduced without fundamentally changing the overall architecture.

It is important to distinguish the representation of an operational rule from the execution of its action. In the current prototype, these rules express the configured responses associated with detected conditions. They do not establish that a physical robot executed braking, reversing, torque adjustment, or hardware-safing actions.

5.5 Event Latching Results

Event latching was evaluated to examine how the CEE handles a condition that remains active across consecutive telemetry evaluations. Without duplicate suppression, an ongoing anomaly could generate repeated history entries, making it difficult to distinguish a single continuing event from multiple separate occurrences.

The implemented mechanism tracks whether a condition is active. When a condition changes from inactive to active, the system can create a new event entry. While the condition remains active, repeated detections of that same condition are suppressed. If the condition becomes inactive and is subsequently activated again, the system can recognise the new activation as another event.

Table 5.4 describes the expected behaviour of the event-latching mechanism.

Test scenario	Expected behaviour
Condition is inactive	No new event is generated for that inactive condition.
Condition becomes active	A new event entry is created.
Condition remains active across repeated evaluations	Duplicate entries for the same ongoing event are suppressed.
Condition becomes inactive	The latch is reset according to the implemented logic.
Condition becomes active again	A new event entry can be created.

Table 5.4: Event-Latching Behaviour

The event-latching mechanism improves the organisation of the event history by preventing repeated records for a continuously active condition. This is particularly useful when telemetry is evaluated repeatedly and a condition remains above or below its threshold for an extended period.

The result demonstrates a mechanism for managing repeated detections, rather than a learning capability. Event latching does not determine new rules, learn from previous anomalies, or independently establish that the underlying physical problem has been resolved.

5.6 Knowledge History and Retention

The knowledge-history component records the operational events identified by the reasoning engine and their associated predefined rule descriptions. This history allows users to inspect detected conditions during the active application session rather than relying exclusively on the current telemetry values.

The history works together with event latching. When a condition becomes active, the event can be recorded. If the same condition remains active, further entries are suppressed. This helps maintain a more readable history of the events observed during the simulation.

Table 5.5 summarises the functions demonstrated by the operational knowledge-history component.

Function	Observed or intended outcome
Event recording	Records detected events during the active session.
Rule association	Connects events with predefined operational rule descriptions.
Duplicate suppression	Reduces repeated entries for continuously active conditions.
History inspection	Makes recorded events available for review.
Session-based retention	Maintains event history during the active session.

Table 5.5: Knowledge-History Functions

The knowledge-history component provides a foundation for reviewing the system's operational events. However, the current implementation maintains an in-session operational history and does not establish durable storage across application restarts. It should not be described as a persistent knowledge database or a formal semantic knowledge graph.

The prototype therefore demonstrates operational event recording and session-level retention. Persistent storage, retrieval across sessions, and the use of historical records to modify future reasoning remain possible areas for future development.

5.7 Dashboard Visualization

The dashboard provides a visual interface for monitoring the simulated robot's telemetry and inspecting the events identified by the CEE. It brings together the information generated by the simulation, telemetry processing, reasoning, and operational history components.

The telemetry displays help the user observe the robot's current simulated state. The event information presents the interpretations produced when predefined conditions are satisfied, while the knowledge-history display allows recorded events to be reviewed during the active session.

The visualization supports the interpretation of system behaviour by making the relationship between telemetry and detected events easier to inspect. For example, the user can examine ultrasonic distance and wheel RPM in relation to the configured obstacle-proximity condition.



Figure 5.4: Telemetry Visualization and Operational Event Display

The dashboard improves the accessibility of the monitoring information and supports functional demonstration of the prototype. Nevertheless, visualization is a presentation mechanism; it does not independently establish that the detected event is physically correct.

Event interpretation remains dependent on the predefined rules and the telemetry supplied to the system.

5.8 Functional Testing

Functional testing was used to examine the behaviour of the main CEE components. The tests focus on whether the implemented workflow performs its intended operations, from telemetry collection to event recording and visualization.

Table 5.6 presents the principal functional test cases and the outcomes that should be verified during execution.

Test area	Verification procedure	Expected result
Telemetry monitoring	Observe the telemetry values produced by the simulator.	Values are available for monitoring and processing.
MQTT communication	Run the configured telemetry publisher and subscriber.	Messages are received when the broker and clients are correctly configured.
Condition-based reasoning	Supply telemetry satisfying a predefined condition.	The corresponding event is identified.
Operational rule mapping	Trigger a configured event.	The event is associated with the intended rule description.
Event latching	Keep an anomaly condition active through repeated evaluations.	Duplicate history entries are suppressed.
Latch reset	Clear a condition and activate it again.	The new activation can be recorded separately.
Knowledge history	Trigger an event and inspect the history.	The recorded event is available during the active session.
Dashboard visualization	Observe the dashboard during system execution.	Available telemetry and event information are displayed.

Table 5.6: Functional Testing Checklist

The tests examine individual components and their interaction within the overall workflow. Testing the conditions independently helps establish whether the correct rule is activated, while repeated evaluations of an active condition help verify duplicate suppression.

Communication testing is also important because MQTT connectivity affects whether telemetry reaches the reasoning component. A communication failure should be investigated separately from the correctness of the rule-evaluation logic.

The table specifies the test procedures and the outcomes expected from each. A test is regarded as passed only when the corresponding behaviour has been verified during execution, and numerical performance measurements are not reported because no such measurements have been made.

5.9 Observed Limitations

Although the CEE prototype demonstrates an integrated workflow for telemetry monitoring, condition-based reasoning, and operational history, several limitations must be considered when interpreting the results.

No.	Limitation	Description
1	Simulation-based evaluation	The prototype uses an interactive mobile-robot simulator. Its behaviour does not establish that the same event-detection logic will perform reliably on a physical robot with real sensors, environmental disturbances, and actuator constraints.
2	Predefined reasoning rules	The system relies on explicitly defined thresholds and logical conditions. It can identify configured events, but it does not autonomously discover new operational rules or learn thresholds from experience.
3	Dependence on telemetry quality	Incorrect, missing, delayed, or implausible telemetry can affect event detection. For example, a zero-valued telemetry state may be interpreted as a possible telemetry failure according to the configured rule, but zero values alone do not prove that communication has failed in every circumstance.
4	Limited knowledge retention	The operational history is maintained during the active session. Durable persistence across application restarts has not been established, limiting the use of historical records across separate sessions.
5	No validated physical actuation	Operational rule descriptions may specify responses such as braking, reversing, or adjusting torque. The prototype does not establish that these responses have been executed and validated through physical robot hardware.
6	Limited quantitative evaluation	Without a documented experimental dataset and measured performance results, the system's accuracy, event-detection latency, communication reliability, and false-positive rate cannot be quantified reliably.
7	Event latching limitations	Event latching suppresses duplicate entries for a continuously active condition. It does not establish the underlying cause of an anomaly or guarantee that the detected condition corresponds to a real-world fault.

Table 5.7: Limitations of the CEE Prototype

These limitations define the current scope of the project and provide a basis for future development. They also distinguish the capabilities demonstrated by the prototype from broader goals such as long-term autonomous adaptation, persistent knowledge management, and physical robot control.

5.10 Overall Discussion

The CEE prototype demonstrates how a modular middleware workflow can connect simulated robot telemetry with deterministic reasoning and operational event recording. The integration of the simulator, MQTT communication, predefined rules, event latching, knowledge history, and dashboard provides a structured way to monitor operating conditions and inspect the resulting interpretations.

A key feature is the separation of telemetry collection from event interpretation. The simulator produces telemetry, MQTT transfers the messages, and the reasoning component evaluates the received values against predefined conditions. This structure helps make the workflow understandable and allows individual components to be tested separately.

The operational rule-mapping component provides an explicit connection between detected conditions and their associated rule descriptions. This improves traceability by allowing the user to inspect not only the telemetry values but also the event interpretations produced by the system.

Event latching further improves the readability of the operational history by reducing duplicate entries when a condition remains active. Its contribution is operational rather than cognitive: it manages event state but does not learn new knowledge or independently improve the reasoning rules.

The knowledge-history component extends the monitoring workflow by retaining event records during the active session. These records can be reviewed through the dashboard, helping the user understand which configured conditions were detected during the simulation. However, session-level history should not be confused with persistent long-term knowledge retention.

Overall, the prototype provides a foundation for further work on cognitive middleware for robotics. Future versions could investigate persistent storage, configurable rule management, improved handling of missing telemetry, more extensive functional testing,

and integration with a physical robot. Any future claim of autonomous learning or improved physical autonomy would require additional implementation and experimental validation.

5.11 Summary

This chapter discussed the results and functional behaviour of the Cognitive Evolution Engine prototype. It described the experimental setup, telemetry monitoring, condition-based event detection, operational rule mapping, event latching, knowledge history, dashboard visualization, functional testing, and observed limitations.

The prototype connects simulated robot telemetry to predefined reasoning rules and records detected events in an in-session operational history. Event latching suppresses duplicate records for continuously active conditions, while the dashboard provides a visual representation of telemetry and detected events.

The evaluation is limited to the demonstrated software workflow and does not establish autonomous rule discovery, durable knowledge persistence across restarts, or validated physical robot control. These limitations identify important opportunities for future work and provide a realistic basis for extending the CEE framework.

The next chapter presents the conclusion and future scope of the Cognitive Evolution Engine project.

CHAPTER 6
LIVE MONITORING TOOL AND DEMONSTRATION

6.1 Overview

The Cognitive Evolution Engine (CEE) live monitoring tool provides an interface for observing the operation of the simulated mobile robot and examining the events identified by the reasoning component. It brings together telemetry monitoring, condition-based event detection, operational rule recording, and dashboard visualization in a single demonstration workflow.

The monitoring process begins with telemetry generated by the interactive mobile-robot simulator. The values are transferred through MQTT to the CEE, where they are evaluated against predefined conditions. When a condition is satisfied, the system identifies the corresponding event and associates it with an operational rule. The event-latching mechanism helps prevent repeated history entries while the same condition remains continuously active.

The dashboard presents the available telemetry and operational event information to the user. This enables the user to observe changes in the simulated robot's state and inspect the events detected during the active session.

The demonstration focuses on the implemented software functionality. It does not represent a validated physical robot-control system, and the recorded operational history is maintained during the active session.

The live monitoring dashboard is the primary interface for observing the CEE prototype during execution. It presents the available robot telemetry and operational information in a visual format, helping the user understand the current simulated state and the events identified by the reasoning component.

The dashboard supports the monitoring of five principal telemetry variables: wheel RPM, IMU acceleration, pitch angle, ultrasonic distance, and system temperature. These variables provide information about the simulated robot's movement, orientation, nearby obstacles, and operating temperature.

In addition to displaying telemetry, the dashboard provides access to the event information produced by the condition-based reasoning component. When a configured condition is satisfied, the corresponding event can be examined alongside the other monitoring

information. The operational history helps the user review the events recorded during the current session.

The dashboard therefore acts as the presentation layer of the CEE framework. It makes the outputs of the telemetry and reasoning components easier to inspect without requiring the user to examine raw MQTT messages or internal processing logic directly.

6.3 Telemetry Demonstration

The telemetry demonstration illustrates how simulated robot data is supplied to the CEE and used for condition-based event detection. During execution, the simulator generates values for the monitored variables, and the MQTT communication component transfers the telemetry to the CEE processing workflow.

The received telemetry is evaluated using the predefined rules. When the relevant threshold or combination of conditions is satisfied, the reasoning component identifies the corresponding event. The resulting event information can then be recorded in the operational history and presented through the dashboard.

Examples of conditions that can be demonstrated include:

- **Thermal overload:** The system temperature exceeds the configured threshold of 75°C.
- **Obstacle proximity:** Ultrasonic distance falls below 25 cm while wheel RPM exceeds 50.
- **Possible wheel slip:** Wheel RPM exceeds 120 while IMU acceleration is below 0.8.
- **Severe inclination:** The absolute pitch angle exceeds 15°.
- **Possible sensor blindness:** Ultrasonic distance exceeds the configured threshold of 900 cm.

These conditions illustrate how individual telemetry variables and combinations of variables are used to identify predefined operating events.

The demonstration should be interpreted as condition-based analysis of simulated telemetry. The detected labels indicate that the configured conditions have been satisfied; they do not independently confirm that a corresponding physical fault has occurred.

6.4 Sample Live Monitoring Output

The sample live monitoring output illustrates how the CEE presents telemetry-related events and their associated operational rule descriptions. When the simulator produces telemetry that satisfies a predefined condition, the reasoning component identifies the event and makes the corresponding information available to the monitoring interface.

The output can be examined to understand the relationship between the observed telemetry, the detected condition, and the associated operational rule. For example, a high system-temperature reading may trigger the thermal-overload rule, while a combination of low ultrasonic distance and high wheel RPM may trigger the obstacle-proximity rule.

The event-latching mechanism also affects the displayed operational history. When a condition remains active across repeated evaluations, duplicate entries for the same ongoing event are suppressed. If the condition becomes inactive and is activated again, a new event can be recorded according to the implemented logic.

The sample output therefore provides a practical view of the CEE workflow, from telemetry reception and condition evaluation to operational event recording and visualization.

The figures in this section show outputs captured from the running CEE prototype. The displayed values and event descriptions are presented as they appeared during execution.

6.5 Summary

This chapter presented the live monitoring tool and demonstration of the Cognitive Evolution Engine. It described the dashboard, telemetry monitoring workflow, condition-based event detection, and sample operational outputs.

The demonstration shows how simulated telemetry is transferred through MQTT, evaluated using predefined rules, and presented through the dashboard. Detected events can be associated with operational rule descriptions and recorded in the in-session history, while event latching suppresses duplicate entries for continuously active conditions.

The live monitoring tool provides a visual means of inspecting the prototype's operation and understanding the relationship between telemetry and detected events. The demonstration remains limited to the implemented simulation-based workflow and does not establish autonomous rule learning, persistent history across restarts, or validated physical robot control.

CHAPTER 7
CONCLUSION AND FUTURE WORK

7.1 Conclusion

The Cognitive Evolution Engine (CEE) project was developed to demonstrate how a cognitive middleware layer can support the monitoring, interpretation, and operational event tracking of a mobile robot in a simulated environment. The primary objective was to establish a framework that connects robot telemetry with condition-based reasoning and presents the resulting events through a monitoring dashboard. The prototype integrates an interactive mobile-robot simulator, MQTT-based telemetry communication, a deterministic reasoning module, an in-session operational knowledge history, and a dashboard for visualizing telemetry and detected events.

The system processes telemetry variables including wheel revolutions per minute (RPM), IMU acceleration, pitch angle, ultrasonic distance, and system temperature. These variables are evaluated against predefined conditions to identify situations such as possible telemetry failure, thermal overload, sensor blindness, obstacle proximity, potential wheel slip, traction loss, and severe inclination. When a configured condition is satisfied, the reasoning module maps the detected event to an intended operational response. This demonstrates how structured condition-based reasoning can help organize robot monitoring information and associate sensor observations with predefined responses.

The implementation also incorporates event latching to suppress repeated entries while an anomaly remains continuously active. When a condition becomes inactive, the latch can reset so that a later activation can be recorded as a new event. The in-session operational knowledge history provides a record of detected conditions and their associated rule descriptions, while the dashboard enables the user to observe telemetry and review the system's outputs. Together, these components demonstrate a practical workflow for connecting telemetry monitoring, rule evaluation, event management, and operational history within a single prototype.

The project establishes a functional foundation for further investigation of cognitive middleware in robotic systems. However, the term cognitive evolution in this implementation refers to structured operational event tracking and rule-based interpretation rather than autonomous learning or automatic discovery of new rules. The current prototype is simulation-based, and the intended responses have not been experimentally validated

through physical robot actuation. Similarly, the operational history is maintained during the active session and is not demonstrated as a durable knowledge store across application restarts.

In conclusion, the CEE prototype demonstrates the feasibility of integrating robot telemetry, deterministic reasoning, event latching, operational knowledge history, and live dashboard monitoring into a unified architecture. It provides a starting point for extending the framework with persistent storage, more advanced reasoning techniques, comprehensive evaluation, and eventual validation on physical robotic platforms.

7.2 Limitations

The current implementation has several limitations that define the scope of the demonstrated results and should be considered when interpreting the project.

- **Simulation-based implementation:** The prototype operates in a simulated mobile-robot environment. Its behaviour does not establish that the same results will occur on a physical robot under real-world operating conditions.
- **Deterministic reasoning:** Event detection depends on predefined conditions and thresholds. The system does not autonomously learn new rules, adapt thresholds through experience, or discover previously unknown relationships between telemetry variables.
- **Threshold dependency:** The effectiveness of event detection depends on the configured thresholds and combinations of sensor values. Conditions that are not represented by the existing rules may not be detected, while unusual or noisy readings may activate a rule without confirming the underlying physical fault.
- **Limited telemetry variables:** The prototype uses wheel RPM, IMU acceleration, pitch angle, ultrasonic distance, and system temperature. Additional information, such as battery health, motor current, localization confidence, and sensor status, could provide a more comprehensive view of robot operation.
- **Session-based operational history:** The knowledge history records operational events during the active session. Durable storage across application restarts, long-term event retrieval, and a formal semantic knowledge graph have not been demonstrated.
- **Event-latching constraints:** Event latching suppresses duplicate entries for a continuously active condition. Although this reduces repetitive records, it does not

independently establish that an event is correctly classified or that the system has learned from previous events.

- **Unvalidated physical responses:** Responses such as braking, reversing, limiting RPM, increasing torque response, activating crawl mode, and entering a hardware-safing state represent intended operational actions associated with rules. Their safe and effective execution by physical hardware has not been experimentally verified.
- **Limited quantitative evaluation:** The project does not establish validated numerical results for detection accuracy, false-positive rate, reasoning latency, throughput, or physical control performance. Such measurements require a structured evaluation using recorded trials and defined performance metrics.

These limitations indicate that the current system should be considered a prototype for telemetry interpretation and operational event management rather than a fully autonomous cognitive controller for physical robots.

7.3 Future Scope

Several improvements can be considered to extend the Cognitive Evolution Engine and improve its applicability to more complex robotic systems.

- **Persistent knowledge storage:** Integrate a database or structured file-based repository to preserve operational events across sessions. This would support long-term event retrieval, historical comparisons, and analysis of recurring anomalies.
- **Adaptive threshold management:** Extend the current fixed-threshold approach with configurable or context-sensitive thresholds. Threshold adjustments should be validated against representative data to avoid reducing detection reliability.
- **Advanced reasoning mechanisms:** Investigate combining deterministic rules with statistical anomaly detection or machine-learning techniques. Such methods could complement predefined rules by identifying patterns that are not explicitly covered by the current configuration.
- **Expanded telemetry integration:** Incorporate additional robot measurements, including battery voltage, motor current, wheel encoders, localization estimates, and communication health. A broader telemetry set could improve the context available to the reasoning module.

- **Physical robot validation:** Deploy the framework on a suitable mobile robot and evaluate the communication pipeline, event detection, and operational responses under controlled conditions. Safety mechanisms should be tested before enabling any autonomous physical action.
- **Improved event prioritization:** Introduce severity levels and event-priority mechanisms to distinguish routine operating conditions from potentially critical anomalies. This could help operators focus on the most important events.
- **Enhanced dashboard functionality:** Add event filtering, historical charts, configurable alert thresholds, event acknowledgements, and export facilities. These features could make the monitoring interface more useful during extended experiments.
- **Comprehensive performance evaluation:** Develop repeatable test scenarios to evaluate detection accuracy, false positives, false negatives, communication delays, processing overhead, and event-latching behaviour. Results should be reported using measured data rather than assumed performance values.
- **Multi-robot support:** Extend the architecture to receive and process telemetry from multiple simulated or physical robots. Robot identifiers, topic organization, and separate event histories would be required to maintain clear monitoring across devices.
- **Safe operational response integration:** Connect the reasoning output to a validated control interface with safety constraints, confirmation mechanisms, and fail-safe behaviour. This would help distinguish informational recommendations from commands that can affect physical robot movement.

These future developments could transform CEE from a simulation-focused monitoring prototype into a more robust middleware platform for persistent operational knowledge management, advanced anomaly analysis, and validated robotic decision support.

7.4 Summary

This chapter presented the conclusion, limitations, and future scope of the Cognitive Evolution Engine project. The prototype integrates simulated robot telemetry, MQTT communication, deterministic condition-based reasoning, event latching, in-session operational knowledge history, and dashboard visualization. It demonstrates a structured

approach to interpreting selected telemetry conditions and associating them with predefined operational responses.

The current implementation remains limited by its reliance on fixed rules, simulation-based evaluation, and session-level event history. Future work can address these limitations through persistent storage, expanded telemetry, advanced reasoning, quantitative testing, and physical robot validation. These developments would provide a path toward a more capable and reliable cognitive middleware framework for robotic applications.

CHAPTER 8

REFERENCES

[1] T. Lesort, V. Lomonaco, A. Stoian, D. Maltoni, D. Filliat, and N. Díaz-Rodríguez, "Continual learning for robotics: Definition, framework, learning strategies, opportunities and challenges," *Information Fusion*, vol. 58, pp. 52–68, 2020.

[2] A. Romero, F. Bellas, and R. J. Duro, "A perspective on lifelong open-ended learning autonomy for robotics through cognitive architectures," *Sensors*, vol. 23, no. 3, Art. no. 1611, Feb. 2023.

[3] M. Beetz, G. Kazhoyan, and D. Vernon, "Robot manipulation in everyday activities with the CRAM 2.0 cognitive architecture and generalized action plans," *Cognitive Systems Research*, vol. 92, Art. no. 101375, Jun. 2025.

[4] M. Lei *et al.*, "RoboMemory: A brain-inspired multi-memory agentic framework for interactive environmental learning in physical embodied systems," arXiv:2508.01415, 2026.

[5] Y. Meng, Z. Bing, X. Yao, K. Chen, K. Huang, Y. Gao, F. Sun, and A. Knoll, "Preserving and combining knowledge in robotic lifelong reinforcement learning," *Nature Machine Intelligence*, vol. 7, pp. 256–269, Feb. 2025.

[6] R. Römer, Y. Zhang, Y. Li, and A. P. Schoellig, "CLARE: Continual learning for vision-language-action models via autonomous adapter routing and expansion," arXiv:2601.09512, 2026.

[7] E. Vorobiov, A. J. Mahmood, S. Rezvani, and R. Chhabra, "ARRC: Advanced reasoning robot control—Knowledge-driven autonomous manipulation using retrieval-augmented generation," arXiv:2510.05547, Oct. 2025.

[8] C. L. Shek, F. M. Tariq, S. Bae, D. Isele, and P. Gupta, "KGLAMP: Knowledge graph-guided language model for adaptive multi-robot planning and replanning," arXiv:2602.04129, 2026.

[9] Y. Feng, J. Han, Z. Yang, X. Yue, S. Levine, and J. Luo, "Reflective planning: Vision-language models for multi-stage long-horizon robotic manipulation," in *Proc. 9th Conf. Robot Learning (CoRL)*, 2025, pp. 1–12.

[10] Z. Lou, K. Xu, Z. Zhou, and R. Xiong, "ExploreVLM: Closed-loop robot exploration task planning with vision-language models," arXiv:2508.11918, Aug. 2025.

[11] E. Fallas Hernández, S. Martínez Alonso, J. A. Becerra Permuy, A. Romero, and R. J. Duro, “Autonomous generation of sub-goals for lifelong learning in robots,” arXiv:2503.18914, Mar. 2025.

[12] Y. Yao *et al.*, “Think small, act big: Primitive prompt learning for lifelong robot manipulation,” arXiv:2504.00420, Jun. 2025.

[13] A. Ayub, C. L. Nehaniv, and K. Dautenhahn, “Interactive continual learning architecture for long-term personalization of home service robots,” arXiv:2403.03462, Mar. 2024.

[14] Y. Long, K. Chen, L. Jin, and M. Shang, “DRAE: Dynamic retrieval-augmented expert networks for lifelong learning and task adaptation in robotics,” in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers)*, Jul. 2025, pp. 23098–23141.

[15] A. Murillo-González and L. Liu, “Action flow matching for continual robot learning,” in *Proc. Robotics: Science and Systems (RSS)*, Jun. 2025.

[16] L. Zhang, L. Qian, R. Li, T. Li, and W. Zhang, “Leveraging multi-modal and historical knowledge graphs for continual robot navigation,” *Visual Intelligence*, vol. 4, no. 15, pp. 1–14, Jun. 2026.

[17] M. Kümpel, M. Scheibl, J.-P. Töberg, V. Hassouna, P. Cimiano, B. Wrede, and M. Beetz, “Everything robots need to know about cooking actions: Creating actionable knowledge graphs to support robotic meal preparation,” *Frontiers in Robotics and AI*, vol. 12, Oct. 2025.

[18] R. Mon-Williams, G. Li, R. Long, W. Du, and C. G. Lucas, “Embodied large language models enable robots to complete complex tasks in unpredictable environments,” *Nature Machine Intelligence*, vol. 7, pp. 592–601, Apr. 2025.

[19] C. Sun, S. Huang, H. Liu, J. Gong, and D. Pompili, “Retrieval-augmented hierarchical in-context reinforcement learning and hindsight modular reflections for task planning with LLMs,” arXiv:2408.06520, Oct. 2024.

[20] M. Gao *et al.*, “Arcadia: Toward a full-lifecycle framework for embodied lifelong learning,” arXiv:2512.00076, Nov. 2025.

CHAPTER 9
APPENDIX

9.1 Telemetry Variable Definitions

The Cognitive Evolution Engine (CEE) uses telemetry data obtained from the simulated mobile robot to monitor its operating conditions. The collected values are passed through the MQTT communication layer to the reasoning module, where they are evaluated against predefined conditions. Each telemetry variable represents a different aspect of robot operation and contributes to identifying potential anomalies.

The telemetry state at time t is represented as:

$$\mathbf{x}_t = [R_t,\ A_t,\ P_t,\ U_t,\ T_t]^T$$

where R_t, A_t, P_t, U_t, and T_t represent wheel RPM, IMU acceleration, pitch angle, ultrasonic distance, and system temperature, respectively.

Variable	Description	Purpose in CEE
Wheel RPM (R_t)	Rotational speed of the robot's wheels	Helps identify excessive speed, low wheel activity, and possible traction-related conditions
IMU Acceleration (A_t)	Acceleration measurement obtained from the simulated inertial measurement unit	Supports the interpretation of movement and possible wheel-slip or collision conditions
Pitch Angle (P_t)	Forward or backward inclination of the robot	Helps identify steep inclinations that may require a crawl-mode response
Ultrasonic Distance (U_t)	Estimated distance between the robot and a nearby obstacle	Supports obstacle detection and identification of unusually large or invalid distance readings
System Temperature (T_t)	Reported system temperature	Helps identify possible thermal overload when the configured threshold is exceeded

Table 9.1: Telemetry Variable Definitions

The telemetry variables are evaluated together where required by the configured rules. For example, a low ultrasonic distance combined with wheel RPM above a predefined threshold can indicate obstacle proximity, while wheel RPM, acceleration, and distance together can be used to identify a possible traction-related condition.

The telemetry values are not treated as definitive proof of a physical fault. Instead, they provide the input conditions used by the prototype's deterministic reasoning module to identify possible operational events.

9.2 Operational Rule Conditions

CEE uses predefined condition-based rules to associate telemetry patterns with operational events and intended responses. The rules are evaluated against incoming telemetry values. When a condition is satisfied, the reasoning module identifies the corresponding event and records the associated rule information in the operational history, subject to the event-latching mechanism.

Rule	Condition	Identified Event	Intended Response
R1	All selected telemetry values are zero	Possible telemetry failure	Trigger hardware-safing mode
R2	System temperature >75° C	Possible thermal overload	Throttle cognitive processing
R3	Ultrasonic distance >900 cm	Possible sensor blindness or invalid reading	Use IMU-based interpretation
R4	Ultrasonic distance <10 cm and IMU acceleration <0	Possible wall collision	Reverse and recalculate
R5	Ultrasonic distance <25 cm and wheel RPM >50	Obstacle proximity	Emergency braking
R6	Wheel RPM >120 and IMU acceleration <0.8	Possible wheel slip	Cap motor RPM by 50%
R7	Wheel RPM <50, IMU acceleration <0.3, and ultrasonic distance >30 cm	Possible traction loss	Increase torque response
R8	Absolute pitch angle >15°	Severe inclination	Enable crawl mode

Table 9.2: Operational Rule Conditions

Note: These are configured prototype rules. The responses describe the intended operational actions associated with each condition; they do not establish that physical robot actuators execute those actions.

The rules illustrate how multiple telemetry variables can be used to interpret possible operating conditions. Some rules depend on a single variable, such as temperature or pitch angle, while others require a combination of sensor values. This approach makes the

reasoning logic explicit and allows the configured thresholds and responses to be reviewed independently.

9.3 Prototype Screenshots

This section contains screenshots of the CEE prototype used to demonstrate the simulator, telemetry monitoring, condition-based reasoning, and dashboard outputs.

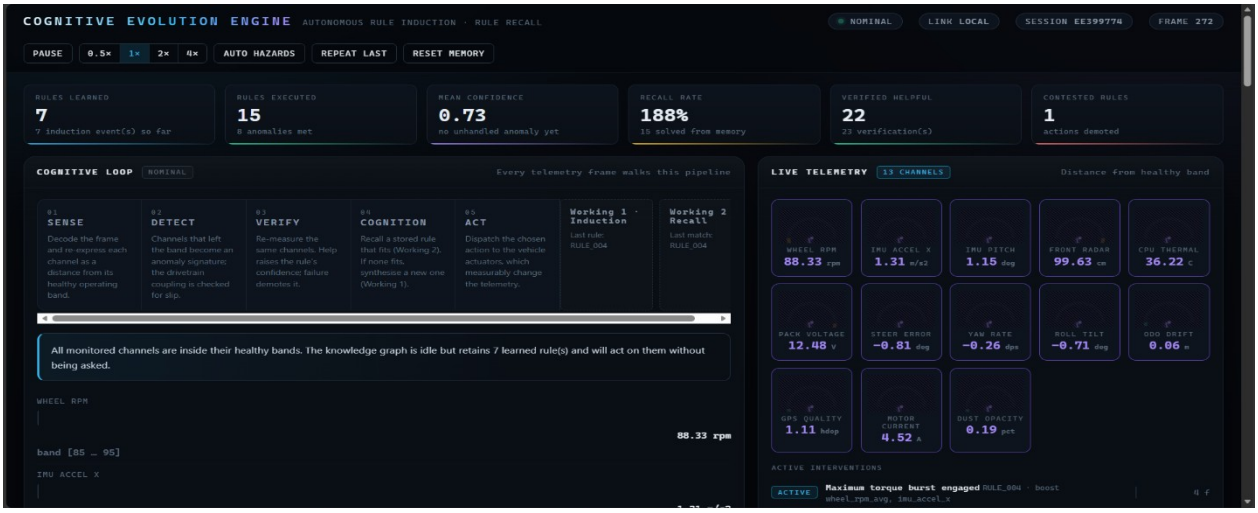


Figure 9.1: Interactive Mobile-Robot Simulator

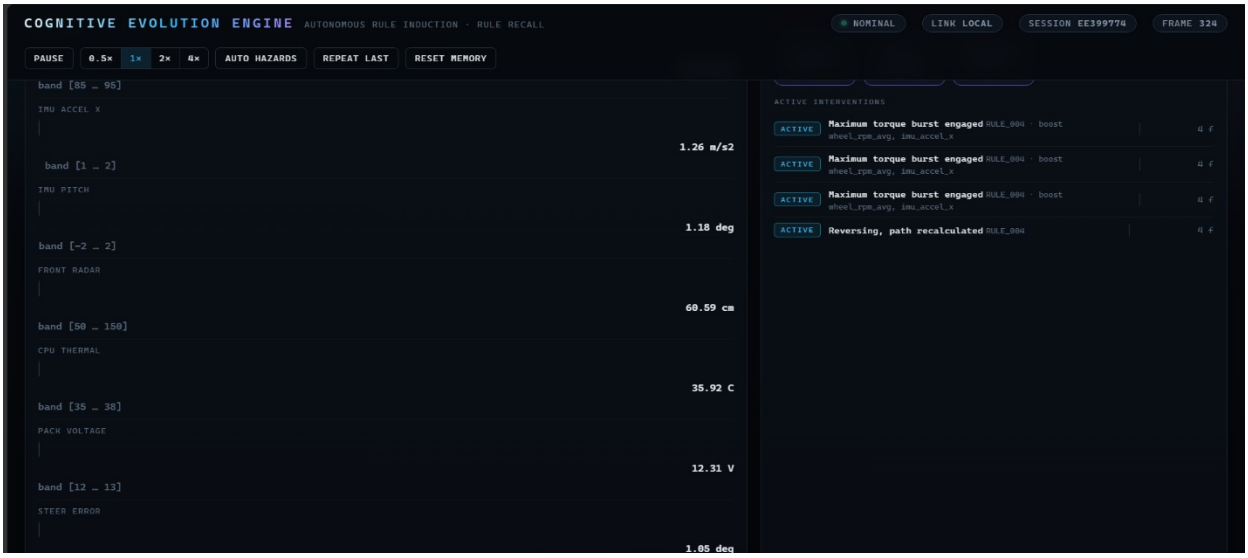


Figure 9.2: Live Telemetry Monitoring Dashboard



Figure 9.3: Condition-Based Event Detection

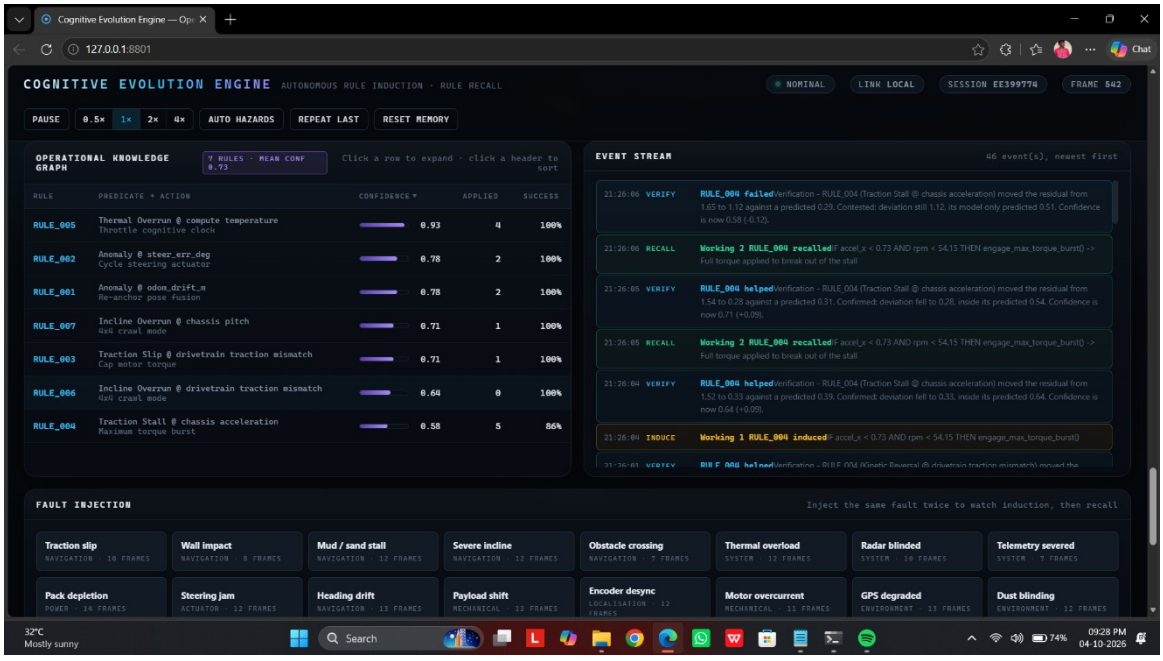


Figure 9.4: Complete CEE Prototype

These screenshots provide visual evidence of the implemented interface and the outputs observed during the demonstration. Each figure is referenced in the relevant sections of Chapters 4, 5, and 6.

CHAPTER 10
SOURCE CODE

10.1 Overview of the Source Code

The source code of the Cognitive Evolution Engine is organized around the main functions required to receive telemetry, evaluate operating conditions, manage event records, and display monitoring information. The implementation connects an interactive mobile-robot simulator with an MQTT communication layer and a condition-based reasoning module. The resulting event information is presented through a dashboard, which allows the user to monitor telemetry and inspect detected conditions.

The source code can be understood through four primary functional components:

1. **Telemetry processing:** Receives and interprets the telemetry variables transmitted by the simulator.
2. **Condition-based reasoning:** Evaluates the telemetry values against predefined operational rules.
3. **Event latching and operational history:** Suppresses duplicate entries while a condition remains active and maintains the event history during the current session.
4. **Dashboard and visualization:** Presents the incoming telemetry and reasoning outputs for live monitoring.

The following subsections describe the role of each component and provide representative pseudocode to explain the processing logic. The pseudocode is illustrative and is not a substitute for the exact implementation files.

10.2 Telemetry Processing

The telemetry-processing component forms the input stage of CEE. It receives the robot's telemetry through the communication pipeline and makes the values available to the reasoning module. MQTT is used to transport telemetry messages between the simulator and the CEE processing component.

The primary telemetry variables are wheel RPM, IMU acceleration, pitch angle, ultrasonic distance, and system temperature. These values form the robot's current telemetry state, which is evaluated whenever a usable update is received.

The processing sequence can be summarized as follows:

1. Receive a telemetry message from the MQTT communication layer.
2. Extract the available telemetry fields.
3. Convert values to the expected data types and verify that they can be interpreted.
4. Update the current telemetry state.
5. Pass the updated state to the condition-based reasoning module.
6. Make the current values available to the dashboard.

```
def process_telemetry(message):
    telemetry = decode_message(message)
    state = {
        "wheel_rpm": telemetry["wheel_rpm"],
        "imu_acceleration": telemetry["imu_acceleration"],
        "pitch_angle": telemetry["pitch_angle"],
        "ultrasonic_distance": telemetry["ultrasonic_distance"],
        "system_temperature": telemetry["system_temperature"]
    }
    update_current_telemetry(state)
    evaluate_operational_rules(state)
    update_dashboard(state)
```

The telemetry-processing component is responsible for supplying the reasoning module with current sensor information. Communication failures, malformed messages, or unavailable values should be distinguished from valid sensor measurements when such handling is supported by the implementation.

10.3 Condition-Based Reasoning

The condition-based reasoning component evaluates the incoming telemetry state against predefined rules. Each rule contains a condition associated with a possible operational event and an intended response. The module uses deterministic logic: the same telemetry state and rule configuration produce the same rule evaluation outcome.

For example, if the system temperature exceeds 75° C, the configured thermal-overload rule can be activated. Similarly, a combination of ultrasonic distance below 25 cm and wheel RPM above 50 can trigger the configured obstacle-proximity event.

Representative pseudocode:

```
def evaluate_operational_rules(state):
    if state_is_all_zero(state):
        register_event("Possible telemetry failure")
    if state["system_temperature"] > 75:
        register_event("Possible thermal overload")
    if state["ultrasonic_distance"] > 900:
        register_event("Possible sensor blindness")
    if (
        state["ultrasonic_distance"] < 10
```

```
        and state["imu_acceleration"] < 0
    ):
        register_event("Possible wall collision")
    if (
        state["ultrasonic_distance"] < 25
        and state["wheel_rpm"] > 50
    ):
        register_event("Obstacle proximity")
    if (
        state["wheel_rpm"] > 120
        and state["imu_acceleration"] < 0.8
    ):
        register_event("Possible wheel slip")
    if (
        state["wheel_rpm"] < 50
        and state["imu_acceleration"] < 0.3
        and state["ultrasonic_distance"] > 30
    ):
        register_event("Possible traction loss")
    if abs(state["pitch_angle"]) > 15:
        register_event("Severe inclination")
```

Once a rule is satisfied, its event information can be passed to the event-management component. The associated response description is used to communicate the intended action. Unless separately connected to a validated control interface, a rule being triggered should not be interpreted as confirmation that the corresponding physical action has occurred.

10.4 Event Latching Logic

The event-latching component manages repeated activations of operational rules. Without latching, a condition that remains true across consecutive telemetry updates could create repeated entries for the same continuously active anomaly. Event latching helps prevent this behaviour by tracking whether an event is already active.

The basic behaviour is:

- When a rule changes from inactive to active, a new event entry can be recorded.
- While the same rule remains active, duplicate entries are suppressed.
- When the rule becomes inactive, its latch can reset.
- If the condition becomes active again after resetting, a new event entry can be created.

```
import json

import time

import networkx as nx

import paho.mqtt.client as mqt

class CognitiveEvolutionEngine:

    def __init__(self, broker_address="127.0.0.1"):
```

```
        print("[CEE] Initializing Cognitive Evolution Engine...")
        # 1. Initialize Operational Knowledge Graph
        self.knowledge_graph = nx.DiGraph()
        self.knowledge_graph.add_node("CEE_Core", type="system"
        # 2. Setup MQTT Communication Bridge
        self.broker_address = broker_address
        self.client = mqtt.Client("CEE_Edge_Node")
        self.client.on_connect = self.on_connect
        self.client.on_message = self.on_message
        # 3. Memory & State
        self.active_rules = {}
    def connect_and_start(self):
        print(f"[CEE] Connecting to MQTT Broker at {self.broker_address}...")
        self.client.connect(self.broker_address, 1883, 60)
        self.client.loop_start()
    def on_connect(self, client, userdata, flags, rc):
        print("[CEE] Connected to Perception Layer (ESP32).")
        # Subscribe to the raw telemetry from the ESP32
        self.client.subscribe("rover/telemetry")
    def on_message(self, client, userdata, msg):
        # Parse incoming JSON telemetry
        try:
            telemetry = json.loads(msg.payload.decode())
            self.experience_interpreter(telemetry)
        except json.JSONDecodeError:
            print("[CEE] Error: Invalid telemetry payload.")
    def experience_interpreter(self, telemetry):
        """
        Translates raw sensory data to find discrepancies (Anomalies)
        """
        wheel_rpm = telemetry.get("encoder_rpm", 0)
        imu_accel = telemetry.get("imu_forward_accel",
```

```
        # Check if the existing rules already handle this state
        if self.decision_engine(telemetry):
            return
       # Anomaly Detection: High wheel spin but low physical acceleration (Wheel Slip)
        if wheel_rpm > 100 and imu_accel < 0.5:
            print("\n[CEE Warning] Anomaly Detected: Intent vs. Action Mismatch!")
            print(f" -> Wheels spinning at {wheel_rpm} RPM, but moving at {imu_accel}
m/s^2")
            anomaly_data = {
                "type": "WHEEL_SLIP",
                "trigger_rpm": wheel_rpm,
                "resulting_accel": imu_accel
            }
            self.reflective_reasoning(anomaly_data)
    def reflective_reasoning(self, anomaly):
        """
        Analyzes the anomaly and generates a structured operational rule
        """
        print("[CEE] Reflective Reasoning Engine engaged..."
        if anomaly["type"] == "WHEEL_SLIP":
            # Generate a new rule: If slipping, cap max RPM to regain traction
            new_rule_id = f"RULE_SLIP_{int(time.time())}"
            new_rule = {
                "condition": "encoder_rpm > 80 and imu_forward_accel < 0.5",
                "action": "set_max_rpm",
                "value": 60,  # Slow down to regain grip
                 "explanation": "High RPM with low IMU accel indicates slip. Reducing
torque."
            }
            print(f"[CEE] New Cognitive Rule Generated: {new_rule['explanation']}")
            self.update_knowledge_graph(new_rule_id, new_rule)
```

```
    def update_knowledge_graph(self, rule_id, rule_data):
        """
        Injects the new rule into the active NetworkX Graph Memory
        """
        self.knowledge_graph.add_node(rule_id, **rule_data)
        self.knowledge_graph.add_edge("CEE_Core", rule_id, relation="derived_rule")
        # Add to active memory for quick lookup in the decision engine
        self.active_rules[rule_id] = rule_data
                    print(f"[CEE]  Knowledge  Graph  Updated.  Total  Rules  stored:
{len(self.active_rules)}")
    def decision_engine(self, telemetry):
        """
       Checks current telemetry against the Knowledge Graph rules to override default
navigation
        """
        wheel_rpm = telemetry.get("encoder_rpm", 0)
        imu_accel = telemetry.get("imu_forward_accel", 0)
        for rule_id, rule in self.active_rules.items():
            # Very basic condition parser for the prototype
          if "encoder_rpm > 80" in rule["condition"] and wheel_rpm > 80 and imu_accel <
0.5:
                print(f"[CEE Decision Engine] Applying {rule_id}: Regulating Speed to
{rule['value']} RPM")
               # Send the physical actuation command back to the ESP32/Motor Controller
                command = {"cmd": rule["action"], "value": rule["value"]}
                self.client.publish("rover/commands", json.dumps(command))
                return True # Rule applied, anomaly averted
        return False # No rule needed, operate normally
# --- Run the Engine ---
if __name__ == "__main__":
    engine = CognitiveEvolutionEngine(broker_address="localhost")
    engine.connect_and_start()
    print("[CEE] Middleware is running. Waiting for physical telemetry...\n"
```

```
    try:
        # Keep the script running
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        print("\n[CEE] Shutting down Cognitive Evolution Engine.")
        engine.client.loop_stop()
```

The latch state is used to distinguish a new event activation from repeated updates belonging to an existing event. This supports a clearer operational history and reduces unnecessary duplication in the dashboard.

Event latching does not independently learn from previous events, change the configured rules, or confirm that an anomaly has been correctly diagnosed. It is an event-management mechanism within the deterministic reasoning workflow.

The operational history described in this project is maintained during the active session. Persistent storage across application restarts is not claimed, because no database or file-based persistence mechanism has been implemented and tested in the prototype.

10.5 Dashboard and Visualization

The dashboard is the user-facing component of CEE. It presents the telemetry received from the simulator and displays the operational information generated by the reasoning module. Its purpose is to make the current robot state and detected events easier to inspect during a simulation session.

The dashboard's main responsibilities include:

- Displaying current telemetry values.
- Visualizing changes in sensor readings where supported by the interface.
- Presenting detected operational events and their associated rule descriptions.
- Showing the operational knowledge or event history maintained during the session.
- Supporting observation of the prototype while the simulated robot is running.

Representative pseudocode:

```
def update_dashboard(telemetry_state):
    display_telemetry(telemetry_state)
```

```
def display_operational_event(event):
    show_event_details(
        event_name=event["name"],
        rule_description=event["rule"]
    )

def update_operational_history(event_history):
    render_event_history(event_history)
```

The dashboard provides visibility into the relationship between telemetry inputs and the outputs of the reasoning module. It is a monitoring and presentation component; it does not independently establish the accuracy of the rule conditions or validate physical robot control.

In summary, the source-code architecture demonstrates how telemetry processing, deterministic condition evaluation, event latching, and dashboard visualization can be organized into a connected workflow. The implementation provides a foundation for future enhancements such as persistent event storage, expanded rule management, automated testing, and integration with a physically validated robot-control system.